WARWICK
THE UNIVERSITY OF WARWICK

Manuscript version: Author's Accepted Manuscript
The version presented in WRAP is the author's accepted manuscript and may differ from the published version or Version of Record.

Persistent WRAP URL:
http://wrap.warwick.ac.uk/184582

How to cite:
Please refer to published version for the most recent bibliographic citation information. If a published version is known of, the repository item page linked to above, will contain details on accessing it.

Copyright and reuse:
The Warwick Research Archive Portal (WRAP) makes this work by researchers of the University of Warwick available open access under the following conditions.

Copyright © and all moral rights to the version of the paper presented here belong to the individual author(s) and/or other copyright owners. To the extent reasonable and practicable the material made available in WRAP has been checked for eligibility before being made available.

Copies of full items can be used for personal research or study, educational, or not-for-profit purposes without prior permission or charge. Provided that the authors, title and full bibliographic details are credited, a hyperlink and/or URL is given for the original metadata page and the content is not changed in any way.

Publisher's statement:
Please refer to the repository item page, publisher's statement section, for further information.

For more information, please contact the WRAP Team at: wrap@warwick.ac.uk.

warwick.ac.uk/lib-publications

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2024.3373722

Constrained Risk-Sensitive Deep Reinforcement Learning for eMBB-URLLC Joint Scheduling

Wenheng Zhang, *Member, IEEE*, Mahsa Derakhshani, *Senior Member, IEEE*, Gan Zheng, *Fellow, IEEE*, and Sangarapillai Lambotharan, *Senior Member, IEEE*

***Abstract*—In this work, we employ a constrained risk-sensitive deep reinforcement learning (CRS-DRL) approach for joint scheduling in a dynamic multiplexing scenario involving enhanced mobile broadband (eMBB) and ultra-reliable low-latency communications (URLLC). Our scheduling policy minimizes the adverse impact of URLLC puncturing on eMBB users while satisfying URLLC requirements. Conventional DRL-based algorithms for eMBB/URLLC scheduling prioritize maximizing the expected return. However, for URLLC mission-critical applications, it is crucial to explicitly avoid catastrophic scheduling failures associated with the long tail of the reward distribution. Therefore, robust management of such uncertainties and risks is imperative. Our proposed CRS-DRL algorithm incorporates the conditional Value-at-Risk (CVaR) as the risk criterion for optimization. A URLLC queuing mechanism is considered to decrease the URLLC drops and increase eMBB throughput compared to the instant scheduling policy. Our architecture is based on the actor-critic model but considers a transfer function to obtain feasible solutions of the unconstrained actor network, and the critic predicts the entire distribution over future returns instead of simply the expectation. Numerical results indicate that our CRS-DRL algorithm, under varying CVaR levels, achieves similar expected returns but reduces long-tail behavior for long-term rewards compared to the risk-neutral approach.**

***Index Terms*—eMBB, deep reinforcement learning, punctured scheduling, resource allocation, risk-sensitive, URLLC.**

I. Introduction

With the rapid development of wireless communication technologies and the increased demands for high-quality digital services, the fifth-generation (5G) and beyond cellular networks have not only provided customers with traditional network services but also met requirements of diversified services [1]–[3]. However, supplying various services within a limited frequency spectrum challenges the traditional wireless networks. For example, the enhanced mobile broadband (eMBB) services aim to reach high utility values to satisfy the quality of service (QoS) [4], while for the ultra-reliable and low-latency communications (URLLC) services, each packet targets to be transmitted within one millisecond with the packet losses being less than 10^{-5} [5]. Thus, it is imperative to investigate how the base station (BS) can wisely choose a scheduling policy to guarantee the different requirements of eMBB and URLLC services in a coexisting system while maximizing the full use of the frequency band.

Different scheduling policies have been developed to optimally solve radio resource allocation in the joint eMBB/URLLC system. Considering the URLLC latency requirement, each URLLC packet cannot wait until the next eMBB slot to be transmitted and thus the idea of scheduling eMBB and URLLC packets on different time scales has been proposed to satisfy the requirements of different services [6]. In particular, each eMBB transmission time slot is divided into several mini slots. The BS immediately schedules each arrived URLLC packet within a short mini slot to minimize its latency and maximize its reliability. Besides, there are reservation-based scheduling schemes proposed (e.g., [7]) wherein the BS reserves a fixed number of resource blocks for URLLC services regardless of eMBB traffic. However, such schemes could lead to under utilization considering random and sporadic nature of URLLC packet arrivals.

Therefore, a more opportunistic approach has been proposed in [8] using a puncturing policy for URLLC scheduling in which the arrived URLLC packet will be scheduled on a mini slot by suspending the on-going eMBB transmission in that resource block. With such a policy, the time-frequency resources will be fully used by eMBB users if there is no URLLC packets. In the following, we review the existing puncturing-based joint dynamic scheduling techniques.

A. Related Works

Many studies have applied optimization methods to design model-based scheduling algorithms for eMBB/URLLC coexistence services on the same radio resources [9]–[15]. For example, the authors in [9] proposed various models for the eMBB rate loss to investigate the impact of URLLC puncturing/superposition on eMBB traffic. In [10], the authors proposed a risk-sensitive formulation that considered a risk measure for eMBB services, constraining the URLLC reliability. [11] applied a penalty-based approach for eMBB user equipments (UEs) and an optimal transportation model for URLLC UEs. Additionally, they propose a heuristic algorithm for scheduling among eMBB UEs. In [12], the authors employed a combination of Difference of Convex programming and Successive Convex Approximation to address the scheduling problem associated with power allocation and user

This work was supported by the UK Engineering and Physical Sciences Research Council (EPSRC) under grant numbers EPSRC EP/X012301/1, EP/X04047X/1, and EP/Y037243/1.

Wenheng Zhang and Mahsa Derakhshani are with Signal Processing and Networks Research Group, Wolfson School, Loughborough University, Loughborough LE11 3TU, U.K. (email: W.Zhang2@lboro.ac.uk; M.Derakhshani@lboro.ac.uk).

Gan Zheng is with the School of Engineering, University of Warwick, Coventry, CV4 7AL, U.K. (email: Gan.Zheng@warwick.ac.uk)

Sangarapillai Lambotharan is with the Institute for Digital Technologies, Loughborough University, E20 3BS London, U.K. (e-mail: s.lambotharan@lboro.ac.uk).

Authorized licensed use limited to: University of Warwick. Downloaded on June 18,2024 at 10:22:54 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2024.3373722

selection, respectively. In [13], the authors considered the URLLC requirements in the medium access control layer and formulated an optimization problem that maximized the utility of the eMBB users subject to a constraint on the latency of URLLC traffic. However, most existing studies adopted the instant scheduling policy for URLLC, i.e., if the incoming URLLC packets can not be transmitted at the next mini-slot, such packets will be immediately dropped as losses. Some studies enhanced the URLLC reliability of the instant scheduling scheme by analyzing the delay of the potential URLLC queuing mechanism. For instance, an M/G/1 queuing model was applied in [14], the queue model applies a packet arrival rate pattern that follows a Poisson distribution, service rate follows a general distribution, and packets are served one by one. The required bandwidth for eMBB and URLLC services was minimized by deriving the closed-form of transmission recovery delay of URLLC traffic. In addition, [15] proposed a queuing mechanism for URLLC traffic and kept the packets in the queue within the latency requirement. In [15], the impact of the queuing latency on the eMBB throughput and URLLC reliability were investigated.

Although the optimization techniques have been useful in solving the model-based joint eMBB/URLLC scheduling problem in 5G communications, there are still some limitations. First, the rapid change of mobile links makes it difficult to accurately capture the diverse and dynamic system state of the eMBB services. In this regard, the authors in [16], [17] assumed that the system states of the eMBB users are known perfectly by the BS. However, in practice, the system states are random and uncertain, and thus significantly impact channel allocation. Therefore, obtaining the stable probability distribution of system parameters in the long term is extremely challenging. Second, considering dynamic scheduling, deriving the closed-form expressions of eMBB and URLLC stochastic performance measures (such as average throughput or packet error rates) could be infeasible and thus model-based optimization methods could become impractical.

Tackling the above limitations of model-based optimization approaches, it is necessary to investigate the model-free optimization and learning methods to dynamically and intelligently allocate the limited resources to eMBB and URLLC traffic through interactions with the environment. Deep reinforcement learning (DRL) algorithms have been appropriate choices to learn a better scheduling decision for the dynamic multiplexing scenario of eMBB-URLLC traffic under uncertainty [18]–[21]. A deep Q-Network (DQN) learning-based resource allocation algorithm was used in [18] for the enormous discrete state space, while the deep deterministic policy gradient (DDPG) based algorithm was proposed in [19], [20] to handle the continuous action space. Besides, [21] addressed the resource allocation problem by applying a DRL policy gradient algorithm, i.e., the proximal policy optimization, where the DRL agent could choose the start time of the URLLC transmission within a URLLC latency constraint and keep the number of punctured eMBB codewords at minimum levels.

B. Motivations and Contributions

For URLLC and eMBB mission critical applications, learning to act in a dynamic environment needs to take into account risks that arise during the learning process due to uncertainties (such as the uncertainty of the distribution of the channel state). Effective management of risks in RL-based scheduling implies mitigating the chance of huge but rare losses that could occur while learning. Most of the existing DRL-based algorithms proposed for joint URLLC and eMBB scheduling (e.g., [18]–[21]) are risk-neutral. In other words, they only focuses on maximizing the expected return and ignore the occurrence of risky situations. The recent work [22] incorporated a risk measure associated with eMBB transmissions using a mean-variance measure. In [22], the risk is only considered for eMBB traffic by maximizing the mean value of eMBB data rate and simultaneously minimizing the variance of eMBB data rate. However, mean-variance risk measure is not very sensitive to the tail behaviour of the long-term rewards distribution [23]. The term 'tail behavior' of the long-term rewards distribution pertains to rare and extreme events that may occur during the operation of a wireless communication system. These events, though low in probability, can have a significant impact, causing scenarios where the system performs exceptionally poorly, leading to disruptions or losses in both URLLC and eMBB services. For instance, such disruptions may manifest as very low probability of no drop for URLLC service (which could lead to substantial packet loss and high latency) as well as very low throughput for eMBB services (which could result in service disruptions). Therefore, understanding the tail behavior is crucial as it helps identify weaknesses in the system performance. Consequently, mitigating the risks associated with these rare events is essential to ensure the robustness and reliability of wireless communication systems.

Additionally, the complexity and computational intensity associated with formulating and solving constrained optimization problems lead many DRL algorithms to omit constraints for computational efficiency [24]. However, neglecting these constraints could result in undesirable consequences, for example, the potentially catastrophic failures. This is especially critical for applications with stringent requirements for reliability and latency, which underscore the necessity for precise constraint management.

To tackle the aforementioned challenges, we have developed a Constrained Risk-Sensitive Deep Reinforcement Learning (CRS-DRL) algorithm. Our goal is to mitigate the risk of enormous loss in URLLC and eMBB services, thereby ensuring a robust scheduling policy for the joint eMBB/URLLC system. To address these risks effectively, we employ the Conditional Value-at-Risk (CVaR) as our chosen risk measure, focusing on the extreme outcomes. The CRS-DRL algorithm maximizes CVaR that is defined as the expectation of long-term rewards only for the region that the cumulative distribution function (CDF) of the long-term rewards is lower than a specific risk level. Specifically, the main contributions of this paper are summarized as follows:

- We model the dynamic joint scheduling problem between

Authorized licensed use limited to: University of Warwick. Downloaded on June 18,2024 at 10:22:54 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2024.3373722

eMBB and URLLC traffic with a queuing mechanism for URLLC as an optimization problem. By determining eMBB resource allocation factors and the URLLC puncturing weights on eMBB users, the scheduling policy aims to maximize the sum of the probability of not dropping URLLC packets and the total normalized eMBB throughout. We introduce a constraint to ensure URLLC outage, defined as the probability of the URLLC channel rate being insufficient to support the required URLLC transmission per mini-slot, remains below a specified threshold.

- We consider an $M/D/B$ queuing mechanism for URLLC traffic to improve the URLLC reliability, whereas the majority of related works [9]–[13], [16]–[22] consider the instant scheduling policy. The proposed queuing mechanism aims to enhance the URLLC reliability within the extreme latency requirement while minimizing the influence on the eMBB utility.
- Given the dynamic nature of the formulated problem and the uncertainties in system state probability distribution, we model the resource allocation problem as a *constrained Markov decision process (CMDP)* problem, and use the Markov's Inequality [10] to transfer the chance constraint into a linear constraint, defining as the cost function. We propose a Constrained Risk-neutral DRL (CRN-DRL) algorithm to solve the constrained optimization problems, inspired by the Lagrange-multiplier method [24], where the cost constraint is integrated to the objective function using Lagrangian multipliers and we address the Lagrangian relaxed problem through a policy network.
- We also propose a CRS-DRL algorithm to learn a suitable scheduling policy that avoids the occurrences of catastrophic scheduling results. The CRS-DRL algorithm, based on an actor-critic architecture inspired by DDPG, but incorporates three key modifications to mitigate long-term rewards/costs risk and reduce the likelihood of low future returns. First, we introduce risk level as an additional input for both actor and critic networks, which is assessed based on satisfying eMBB and URLLC requirements. Second, we include two critic networks to learn the parameters for the long-term rewards and costs functions in CMDP problem, respectively. Third, we adapt the critic networks in the CRS-DRL approach to govern the distribution of future returns rather than providing a scalar expectation, as in standard DDPG. Specifically, the critic networks predict and output the mean and variance of the long-term rewards and costs distributions, facilitating the derivation of the CVaR in each learning step.
- We validate the proposed CRS-DRL algorithm through simulations, comparing it with the CRN-DRL algorithms and a reference method [20], which lacks a queuing mechanism and risk-sensitive considerations. The results focus on highlighting the robustness of CRS-DRL. Notably, we observe a significant reduction in estimated variance compared to the CRN-DRL approach, while the difference in the mean value of long-term rewards remains minimal.

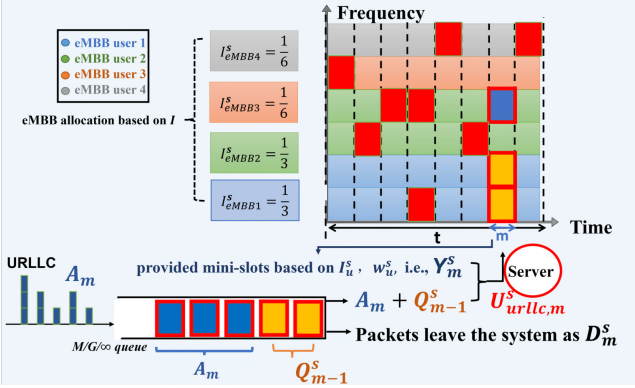


Fig. 1: Illustration of scheduling policy for the eMBB-URLLC coexisting system on the same time-frequency spectrum, in system state $\boldsymbol{s} \in \mathcal{S}$. The served URLLC packets are shown in red rectangles at each mini-slot. The detail of the proposed URLLC queuing mechanism will be presented in Section II-B, with main notations presented in Table I.

TABLE I: Summary of Notations

Notation	Definition
B	Number of resource blocks.
$\mathcal{M}$	Set of mini-slots.
$\mathcal{U}$, U_{embb}	Set of and number of eMBB users, respectively.
m, t	URLLC mini-slot and eMBB time slot, respectively.
λ	URLLC arrival rate.
$\mathcal{C}_{embb}$	Set of possible average channel rates for eMBB users.
$\mathcal{S}$	Set of possible system state.
N_s	Number of possible combinations for system state.
$P_S(\boldsymbol{s})$	PMF of system state $\boldsymbol{s}$.
$\boldsymbol{I}$	Matrix of eMBB allocation factors
$\boldsymbol{W}$	Matrix of URLLC puncturing weights.
A_m, $Y^{\boldsymbol{s}}_m$	Number of arrived URLLC packets and mini-slots allocated to URLLC in $m \in \mathcal{M}$, $\boldsymbol{s} \in \mathcal{S}$, respectively.
$\mathcal{Q}^{\boldsymbol{s}}_{m-1}$	Number of URLLC packets already stored in the queue before $m \in \mathcal{M}$, and in $\boldsymbol{s} \in \mathcal{S}$.
$D^{\boldsymbol{s}}_m$, $U^{\boldsymbol{s}}_{\text{urllc},m}$	Number of dropped and served URLLC packets in $m \in \mathcal{M}$ and $\boldsymbol{s} \in \mathcal{S}$, respectively.
$b(\cdot)$	Bernoulli generate function for URLLC puncturing.
ϵ	Bernoulli generate variable for URLLC puncturing.
p_a, $p^{\boldsymbol{s}}_Q$, $p^{\boldsymbol{s}}_c$	The PMF of A_m, $Q^{\boldsymbol{s}}_{m-1}$ and $C^{\boldsymbol{s}}_m$, respectively.
$\boldsymbol{\Psi}^{\boldsymbol{s}}$	State transition probability of URLLC queue in $\boldsymbol{s} \in \mathcal{S}$.

C. Organization

The remainder of this paper is organized as follows. Section II presents the system model, the working principle of the URLLC queuing mechanism. Section III introduces the proposed problem formulation and the performance metric. In Section IV, we transform the eMBB-URLLC joint scheduling problem as an CMDP problem and solve the optimization problem by CRN-DRL algorithm. The full details of the proposed CRS-DRL algorithm are explained in Section V. Finally, Section VI and VII presents the simulation results and provide concluding remarks, respectively.

Notations: We use $\mathbb{E}[.]$ and $\mathbb{P}[.]$ to denote the expectation and the probability respectively. $\mathbb{N}$ represents the set of natural numbers. The boldface capital and lower case letters are used to represent the matrices and vectors, respectively.

Authorized licensed use limited to: University of Warwick. Downloaded on June 18,2024 at 10:22:54 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2024.3373722

II. System Model

A. eMBB and URLLC Coexistence

In this paper, we consider a downlink network, where the BS jointly and dynamically schedules the coexisting eMBB and URLLC services on B resource blocks under different transmission time intervals. Each eMBB packet is served in a time slot which consists of M mini-slots for URLLC transmission, i.e., mini-slot $m \in \mathcal{M} = \{1, 2, \ldots, M\}$. We assume there is a set of U_{eMBB} eMBB users (denoted by $\mathcal{U}$) in the network, and a single URLLC user for which packets arrive randomly in each mini-slot based on the Poisson distribution with the average arrival rate of λ packets/mini-slot. The set of possible average channel rates for eMBB users is denoted by $\mathcal{C}_{\text{eMBB}} = \{1, 2, \ldots, C_{\text{eMBB}}\}$ and the set of possible average channel rates for the URLLC user is denoted by $\mathcal{C}_{\text{uRLLC}} = \{1, 2, \ldots, C_{\text{uRLLC}}\}$. The wireless system undergoes channel variations in each time slot, and we model the system state in each time slot as a $1 \times (U_{\text{eMBB}} + 1)$ vector : $\boldsymbol{s} = [s_{\text{r}}, \boldsymbol{s}_{\text{e}}]$, where $s_{\text{r}} \in \mathcal{C}_{\text{URLLC}}$ denotes the average channel rate of the URLLC user and $\boldsymbol{s}_{\text{e}} = [s_1, ...s_u, ..., s_{U_{\text{eMBB}}}]$, $s_u \in \mathcal{C}_{\text{eMBB}}$ denoting the average channel rate of eMBB user $u \in \mathcal{U}$. The set of possible system state is denoted by $\mathcal{S} = \{\boldsymbol{s}_1, \cdots, \boldsymbol{s}_{N_s}\}$, where $N_s = {C_{\text{eMBB}}}^{U_{\text{eMBB}}} \times C_{\text{uRLLC}}$, is the number of possible combinations of system state $\boldsymbol{s}$. The probability mass function (PMF) of system state $\boldsymbol{s}$ is $P_S(\boldsymbol{s}) \in [0, 1], \forall \boldsymbol{s} \in \mathcal{S}$. Ideally the system state for eMBB service is a continuous random variable, however, in practical systems the channel quality feedback from users are quantized to several discrete levels. Accordingly, we assume that system state can take finite number of levels considering the continuous model and quantization, although the number of levels can be very large to achieve a certain precision. This assumption facilitates formulation of a stochastic scheduling.

At the start of each time slot, the BS chooses what proportion of the resource blocks to allocate to each eMBB user $u \in \mathcal{U}$ considering the system state $\boldsymbol{s} \in \mathcal{S}$ and fixes such proportion for the duration of a time slot. This sharing mechanism is modeled based on a $N_s \times U_{\text{eMBB}}$ allocation factor matrix $\boldsymbol{I} = [I_u^{\boldsymbol{s}}]$, where $I_u^{\boldsymbol{s}} \in [0, 1]$, represents the fraction of resource blocks allocated to eMBB user $u \in \mathcal{U}$ in system state $\boldsymbol{s} \in \mathcal{S}$. It is assumed that without puncturing the total resources are always fully occupied by the eMBB users in each time slot for each system state, i.e. $\sum_{u \in \mathcal{U}} I_u^{\boldsymbol{s}} = 1$, $\forall \boldsymbol{s} \in \mathcal{S}$.

The illustration of the downlink transmission system is given in Fig. 1 along with Table I listing the main notations used in this work. As shown in Fig. 1, the puncturing scheduling, proposed by [8], is employed in this paper for URLLC. Under this scheme, a portion of the resource blocks allocated to eMBB users is reassigned to URLLC in each mini-slot. The weights of puncturing eMBB users for each system state is denoted as a $N_s \times U_{\text{eMBB}}$ matrix, i.e., $\boldsymbol{W} = [w_u^{\boldsymbol{s}}]$, where $w_u^{\boldsymbol{s}} \in [0, 1]$, indicates the weight of puncturing the eMBB user $u \in \mathcal{U}$ in system state $\boldsymbol{s} \in \mathcal{S}$.

The realization of puncturing weights is random based on the variable $\epsilon_u^{\boldsymbol{s}}$, which is generated using a Bernoulli distribution with the success probability of $w_u^{\boldsymbol{s}}$ at the beginning of each mini-slot as

$$\epsilon_u^{\boldsymbol{s}} = b(w_u^{\boldsymbol{s}}) = \begin{cases} 1, \text{happens with probability } w_u^{\boldsymbol{s}}, \\ 0, \text{happens with probability } 1 - w_u^{\boldsymbol{s}}. \end{cases} \tag{1}$$

If $\epsilon_u^{\boldsymbol{s}} = 1$, URLLC packets can be overridden upon the scheduled $BI_u^{\boldsymbol{s}}$ resource blocks of eMBB user u. We assume each URLLC packet requires one resource block to transmit in each mini-slot. Otherwise, when $\epsilon_u^{\boldsymbol{s}} = 0$, the transmission of eMBB user u will continue with no interruption in that mini-slot. Consequently, the number of provided resource blocks for URLLC in mini-slot $m \in \mathcal{M}$ in system state $\boldsymbol{s} \in \mathcal{S}$ is derived as

$$Y_m^{\boldsymbol{s}} = \sum\nolimits_{u \in \mathcal{U}} BI_u^{\boldsymbol{s}} \epsilon_u^{\boldsymbol{s}}. \tag{2}$$

Suppose the PMF of the total number of provided resource blocks for URLLC in the mini-slot $m \in \mathcal{M}$ is denoted by $P_Y^{\boldsymbol{s}}(y) = \mathbb{P}(Y_m^{\boldsymbol{s}} = y), \forall y \in \{0, \ldots, B\}$. In system state $\boldsymbol{s} \in \mathcal{S}$, $P_Y^{\boldsymbol{s}}(y)$ can be derived as a function of puncturing weights as

$$P_Y^{\boldsymbol{s}}(y) = \sum\nolimits_{\mathcal{E}_y^{\boldsymbol{s}} \in \mathcal{F}_y^{\boldsymbol{s}}} \prod\nolimits_{i \in \mathcal{E}_y^{\boldsymbol{s}}} w_i^{\boldsymbol{s}} \prod\nolimits_{j \in \mathcal{E}_y'^{\boldsymbol{s}}} (1 - w_j^{\boldsymbol{s}}), \tag{3}$$

where the set $\mathcal{F}_y^{\boldsymbol{s}}$ contains the possible combinations of eMBB users that can totally provide y out of B resource blocks in a mini-slot for URLLC puncturing in system state $\boldsymbol{s} \in \mathcal{S}$. Mathematically, $\mathcal{F}_y^{\boldsymbol{s}} = \{\mathcal{E}_y^{\boldsymbol{s}} | \sum_{i \in \mathcal{E}_y^{\boldsymbol{s}}} BI_i^{\boldsymbol{s}} = y\}$ where $\mathcal{E}_y^{\boldsymbol{s}}$ represents a subset of eMBB users, $i \in \mathcal{E}_y^{\boldsymbol{s}}$ represents the eMBB users whose transmissions are punctured by URLLC packets. $\mathcal{E}_y'^{\boldsymbol{s}}$ denotes complementary set of $\mathcal{E}_y^{\boldsymbol{s}}$, $j \in \mathcal{E}_y'^{\boldsymbol{s}}$ represents the eMBB users that keep their transmissions without being punctured. For each combination, the equation calculates the product of puncturing weights w_i^s for eMBB users i in $\mathcal{E}_y^{\boldsymbol{s}}$ and the product of non-puncturing weights w_j^s for eMBB users j in $\mathcal{E}_y'^{\boldsymbol{s}}$. The equation sums up these probabilities over all possible combinations of eMBB users that can collectively provide y resource blocks for URLLC in the current system state $\boldsymbol{s}$. According to (3), note that $P_Y^{\boldsymbol{s}}$ is a constant variable across different mini-slots for a typical time slot, which only depends on the puncturing weights and eMBB allocation factors over time slots rather than mini-slot m in system state $\boldsymbol{s} \in \mathcal{S}$.

Although $Y_m^{\boldsymbol{s}}$ determines the number of provided resource blocks for URLLC transmission, the actual number of punctured resource blocks in a mini-slot for each system state also depends on the number of arrived and queued URLLC packets, which is related to the proposed URLLC queuing mechanism. The detailed analysis of the number of actual served URLLC packets is provided in the next subsection.

B. URLLC Queuing Mechanism

To improve the URLLC reliability of the instant scheduling policy in [9]–[13], [16]–[22]. We have designed an $M/D/B$ URLLC queuing mechanism which allows URLLC packets to remain in the queue until the next mini-slot if they cannot be transmitted immediately. Considering the recommended setting by 3GPP with each time-slot of one millisecond and containing $M = 7$ mini-slots, this queuing policy will maintain the latency less than the target of 0.3 milliseconds [25]. For the $M/D/B$ queuing model, M means Poisson-distributed URLLC arrivals. D stands for deterministic service

Authorized licensed use limited to: University of Warwick. Downloaded on June 18,2024 at 10:22:54 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2024.3373722

times, indicating that each URLLC packet is served for a constant and predictable duration, reflecting the time required for transmission through the channel. B defines the maximum number of servers available at the BS for each decision slot.

The working principle of the queue can be seen in Fig. 1. In mini-slot $m \in \mathcal{M}$ for a typical time slot and system state $\boldsymbol{s} \in \mathcal{S}$, the system considers the already queued URLLC packets (denoted by $\mathcal{Q}^{\boldsymbol{s}}_{m-1}$) as the priority group to serve, and the arrived URLLC packets between mini-slot $m-1$ and mini-slot m as the secondary group (denoted by A_m). Our queuing policy is that at the start of each mini-slot, the BS transmits the priority group packets first, and uses the remaining resource blocks to transmit the secondary group packets. If the priority group packets cannot be served in the mini-slot m, such packets will be dropped as losses. Subsequently, the number of actual served packets (i.e., denoted by $U^{\boldsymbol{s}}_{\text{urllc},m}$) and dropped packets (i.e., denoted by $D^{\boldsymbol{s}}_m$) in mini-slot $m \in \mathcal{M}$ for a typical time slot in system state $\boldsymbol{s} \in \mathcal{S}$ can be obtained as

$$U^{\boldsymbol{s}}_{\text{urllc},m} = \min(Y^{\boldsymbol{s}}_m, \mathcal{Q}^{\boldsymbol{s}}_{m-1} + A_m), \tag{4}$$

$$D^{\boldsymbol{s}}_m = \max(\mathcal{Q}^{\boldsymbol{s}}_{m-1} - U^{\boldsymbol{s}}_{\text{urllc},m}, 0). \tag{5}$$

Accordingly, in system state $\boldsymbol{s} \in \mathcal{S}$ the queuing state in mini-slot $m+1$ is updated as

$$\mathcal{Q}^{\boldsymbol{s}}_m = Q^{\boldsymbol{s}}_{m-1} + A_m - (U^{\boldsymbol{s}}_{\text{urllc},m} + D^{\boldsymbol{s}}_m). \tag{6}$$

III. Problem Formulation

In this paper, we consider a class of joint eMBB-URLLC scheduling policies represented by Π that combines possibly system state dependent eMBB resource allocations and URLLC demands placements per time slot. Thus, a joint scheduling policy $\pi \in \Pi$ is characterized as 1) the eMBB allocation factor matrix $\boldsymbol{I}^{\pi}$; 2) the weights of URLLC puncturing on eMBB users $\boldsymbol{W}^{\pi}$. Suppose there exists an unknown optimal scheduling policy for the eMBB resource allocation and the URLLC preemption, denoted by π^*, that can maximize both the total eMBB utility and the URLLC reliability (which is defined as the probability of not dropping URLLC packets in this paper). Thus, the dynamic resource allocation problem for finding the optimal joint eMBB/URLLC scheduling policy π^* can be formulated as

$$\max_{\pi\in\Pi} \sum_{\boldsymbol{s}\in\mathcal{S}} P_S(\boldsymbol{s}) \big(r^{\pi,\boldsymbol{s}}_{\text{eMBB}} + r^{\pi,\boldsymbol{s}}_{\text{urllc}}\big) \tag{7}$$

$$\begin{aligned}
\textbf{s.t.}\quad & \textbf{C1:}\quad I^{\pi,\boldsymbol{s}}_u \in [0,1], \quad \forall \boldsymbol{s} \in \mathcal{S}, \forall u \in \mathcal{U},\\
& \textbf{C2:}\quad w^{\pi,\boldsymbol{s}}_u \in [0,1], \quad \forall \boldsymbol{s} \in \mathcal{S}, \forall u \in \mathcal{U},\\
& \textbf{C3:}\quad \sum\nolimits_{u\in\mathcal{U}} I^{\pi,\boldsymbol{s}}_u = 1, \quad \forall \boldsymbol{s} \in \mathcal{S},\\
& \textbf{C4:}\quad \mathbb{P}\left(s_r < \frac{\sum_{m\in\mathcal{M}} U^{\pi,\boldsymbol{s}}_{\text{urllc},m}}{M}\right) \le \zeta, \forall \boldsymbol{s} \in \mathcal{S}.
\end{aligned}$$

where $r^{\pi,\boldsymbol{s}}_{\text{urllc}}$ and $r^{\pi,\boldsymbol{s}}_{\text{eMBB}}$ denote the probability of not dropping URLLC packets and the total normalized eMBB throughput in system state $\boldsymbol{s} \in \mathcal{S}$ under the policy π, respectively. **C1** and **C2** guarantee the values of the eMBB allocation factor $I^{\pi,\boldsymbol{s}}_u$ and puncturing weight $w^{\pi,\boldsymbol{s}}_u$ are between zero and one, for each eMBB $\boldsymbol{u} \in \mathcal{U}$ in any system state $\boldsymbol{s} \in \mathcal{S}$ under the policy π. **C3** ensures the total resources to be always fully occupied by the eMBB users in each time slot for each system state without URLLC preemption. **C4** ensures the outage probability of the channel rate is falling below the minimum required URLLC transmitted packets per mini-slot, remains below the specified threshold ζ given the assumption that the transmission of a URLLC packet can be always successful.

In this paper, we are focusing on a stochastic optimization allocation problem to find the optimal strategy for any state, rather than optimizing allocation for instantaneous system state needing real-time optimization. This makes our solution appealing from practical point of view avoiding the need for running optimization every coherence time. On the other hand, the number of states is a challenge for a stochastic optimization and considering a matrix for RB allocation would hinder efficiently solving the stochastic optimization. We have chosen to simplify the system model to make it more suitable for our specific research objectives.

The objective function in (7) considers two metrics to show the performance of the dynamic resource allocation policy π, i.e., $r^{\pi,\boldsymbol{s}}_{\text{urllc}}$ and $r^{\pi,\boldsymbol{s}}_{\text{eMBB}}$, which are analytically derived in the following.

A. URLLC Performance Metric

Studying the queue dynamics, the probability of not dropping URLLC packets in system state $\boldsymbol{s} \in \mathcal{S}$ under the policy π is analysed in Appendix A, and therein the expression is obtained as

$$r^{\pi,\boldsymbol{s}}_{\text{urllc}} = \mathbb{P}\big(D^{\pi,\boldsymbol{s}}_m = 0\big) = \sum_{n=0}^{B} P^{\pi,\boldsymbol{s}}_Q(n) \sum_{y=n}^{B} P^{\pi,\boldsymbol{s}}_Y(y), \tag{8}$$

where $\mathbb{P}\big(D^{\pi,\boldsymbol{s}}_m = 0\big)$ shows the probability that there is no dropped URLLC packet in system state $\boldsymbol{s} \in \mathcal{S}$ for $m \in \mathcal{M}$ under the policy π. $P^{\pi,\boldsymbol{s}}_Y(y), \forall y \in \{0,\dots,B\}$ has been introduced in (3), which represents the PMF of the provided resource blocks for URLLC. Moreover, $P^{\pi,\boldsymbol{s}}_Q(q)$, $q \in \mathbb{N}$ denotes the steady state probability of the URLLC queue in system state $\boldsymbol{s} \in \mathcal{S}$ under the policy π. $P^{\pi,\boldsymbol{s}}_Q(q)$ can be derived as [26]

$$P^{\pi,\boldsymbol{s}}_Q(q) = F(\mathbf{0}_{q,q})[F(\boldsymbol{\Psi}^{\boldsymbol{\pi},\boldsymbol{s}} - \mathbb{I})]^{-1}, \tag{9}$$

where the output of the function $F(\mathbf{A})$ is the matrix $\mathbf{A}$ with its right-most column replaced with all 1's. $\mathbb{I}$ is the identity matrix of size q, and $\mathbf{0}_{q,q}$ denotes the zero matrix of size $q \times q$. Moreover, $\boldsymbol{\Psi}^{\boldsymbol{\pi},\boldsymbol{s}} = [\psi^{\pi,\boldsymbol{s}}_{n,k}]$, $\psi^{\pi,\boldsymbol{s}}_{n,k} = \mathbb{P}(\mathcal{Q}^{\pi,\boldsymbol{s}}_m = k | \mathcal{Q}^{\pi,\boldsymbol{s}}_{m-1} = n), \forall n,k \in \mathbb{N}$, is the state transition probability matrix. The transition probabilities of the URLLC queuing mechanism in state $\boldsymbol{s} \in \mathcal{S}$ under the policy π can be computed as [27]:

$$\psi^{\pi,\boldsymbol{s}}_{n,k} = \tag{10}$$

$$\begin{cases}
P_a(k) \sum\limits_{x=0}^{\min\{B,n\}} P^{\pi,\boldsymbol{s}}_y(x) + \sum\limits_{\theta=1}^{B-n} \Big(\sum\limits_{a_1=k}^{\theta} P_a(a_1) \Big) P^{\pi,\boldsymbol{s}}_y(n+\theta), \\
\text{if } k = 0, \quad \forall n \in \mathbb{N}, \\
P_a(k) \sum\limits_{x=0}^{\min\{B,n\}} P^{\pi,\boldsymbol{s}}_y(x) + \sum\limits_{\theta=1}^{B-n} P_a(k+\theta) P^{\pi,\boldsymbol{s}}_y(n+\theta), \\
\text{if } \forall k \in \mathbb{N}_+, \quad \forall n \in \mathbb{N},
\end{cases}$$

Authorized licensed use limited to: University of Warwick. Downloaded on June 18,2024 at 10:22:54 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2024.3373722

where $P_a(k) = \mathbb{P}(A_m = k) \in [0,1], \forall k \in \mathbb{N}$ denotes the PMF of the number of URLLC packet arrivals and hence $\sum_{k\in\mathbb{N}} P_a(k) = 1$.

B. eMBB Performance Metric

The total normalized eMBB throughput in (7) for system state $\boldsymbol{s} \in \mathcal{S}$ under the policy π can be expressed as

$$r_{\text{eMBB}}^{\pi,\boldsymbol{s}} = T_a^{\pi,\boldsymbol{s}} / T_t^{\pi,\boldsymbol{s}}, \tag{11}$$

where $T_t^{\pi,\boldsymbol{s}}$ is the total eMBB throughput without being punctured by URLLC. $T_a^{\pi,\boldsymbol{s}}$ denotes the total effective eMBB throughput in system state $\boldsymbol{s} \in \mathcal{S}$ under the policy π.

Here, we first analyze $T_t^{\pi,\boldsymbol{s}}$ as the summation of channel rates of all eMBB users considering their resource allocation factors. Thus, we have

$$T_t^{\pi,\boldsymbol{s}} = \sum\nolimits_{u\in\mathcal{U}} s_u I_u^{\pi,\boldsymbol{s}}. \tag{12}$$

where s_u denotes the average channel rate of eMBB user u when all the resource blocks are assigned to eMBB user u without URLLC puncturing.

Besides, $T_a^{\pi,\boldsymbol{s}}$ can be calculated as

$$T_a^{\pi,\boldsymbol{s}} = \sum_{u\in\mathcal{U}} s_u I_u^{\pi,\boldsymbol{s}} \Big(1 - \frac{w_u^{\pi,\boldsymbol{s}} I_u^{\pi,\boldsymbol{s}} \mathbb{E}\big[U_{\text{urllc},m}^{\pi,\boldsymbol{s}}\big]}{\sum_{u'\in\mathcal{U}} w_{u'}^{\pi,\boldsymbol{s}} I_{u'}^{\pi,\boldsymbol{s}}}\Big), \tag{13}$$

where $\mathbb{E}\big[U_{\text{urllc},m}^{\pi,\boldsymbol{s}}\big]$ denotes the expected served URLLC packets in mini-slot $m \in \mathcal{M}$ for a typical time slot in system state $\boldsymbol{s} \in \mathcal{S}$ under the policy π. $w_u^{\pi,\boldsymbol{s}} I_u^{\pi,\boldsymbol{s}} \mathbb{E}\big[U_{\text{urllc},m}^{\pi,\boldsymbol{s}}\big] / \sum_{u'\in\mathcal{U}} w_{u'}^{\pi,\boldsymbol{s}} I_{u'}^{\pi,\boldsymbol{s}}$ represents the fraction of resource blocks that eMBB user u contributes to URLLC packets in mini-slot $m \in \mathcal{M}$ for a typical time slot under the policy π.

To obtain $T_a^{\pi,\boldsymbol{s}}$, first we need to analyse $\mathbb{E}\big[U_{\text{urllc},m}^{\pi,\boldsymbol{s}}\big]$. Based on the analysis in Appendix B, $\mathbb{E}\big[U_{\text{urllc},m}^{\pi,\boldsymbol{s}}\big]$ is expressed as

$$\begin{aligned}\mathbb{E}\big[U_{\text{urllc},m}^{\pi,\boldsymbol{s}}\big] = \sum_{x=0}^{B} x \Bigg(& P_Y^{\pi,\boldsymbol{s}}(x)\big(1 - \sum_{i=0}^{x-1}\sum_{j=0}^{i} P_a(j) P_Q^{\pi,\boldsymbol{s}}(i-j)\big) \\ & + \sum_{j=0}^{x} P_a(j) P_Q^{\pi,\boldsymbol{s}}(x-j) \sum_{i=x+1}^{B} P_Y^{\pi,\boldsymbol{s}}(i)\Bigg).\end{aligned} \tag{14}$$

The above analysis indicates that the expressions of the performance metrics in (8) and (11) contain complex inverse matrix calculation. Therefore, deriving the closed-form expressions of the performance metrics is challenging. Besides, in practice, the stationary distribution of the system state (i.e., $P_S(\boldsymbol{s})$ in (7)) is uncertain to the BS, and a sequential decisions also need to be made over multiple eMBB transmission time slots to obtain the distribution of system state in a dynamic environment. Thus, instead of dealing with the traditional model-based optimization methods, we propose both CRN-DRL and CRS-DRL algorithms to solve this specific optimization problem, the details of which are described in the following two sections, respectively.

IV. CRN-DRL-based Joint eMBB-URLLC Scheduling Algorithm

Before taking into account the risk in the scheduling problem, in this section, we first design a constrained risk-neutral DRL algorithm (CRN-DRL) to solve the eMBB-URLLC joint scheduling optimization problem with queuing mechanism in (7) when $P_S(\boldsymbol{s})$ is unknown. As one of the most trending branch of machine learning methods, DRL has been widely used in finding optimal scheduling decisions under uncertain conditions. It is a combination of reinforcement learning (RL) and deep learning that enhances the learning efficiency of RL with DNNs [28]. At each decision time step, the agent of DRL interacts with the unknown environment by choosing a suitable action in a given state and responding to a corresponding reward, cost and a new state. Besides, we consider to control the URLLC outage as constraint **C4**, which need to be mapped to the cost functions. Based on the Lagrange-multiplier method [24], where the cost constraint is integrated to the objective function using Lagrangian multipliers and we address the Lagrangian relaxed problem through a policy network. The Lagrangian multipliers are updated by the sampled policy gradient in loss functions. Thus, we model the eMBB-URLLC scheduling problem as a CMDP [24], and propose the CRN-DRL to solve the problem.

A. Key Elements for CMDP

In the eMBB-URLLC scheduling problem, the agent is the BS who interacts with a stochastic environment E by sequentially choosing an action in time slots. The CMDP consists of a *state space* $\mathcal{S}$, an *action space* $\mathcal{A}$, the scalar reward function $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$, and the cost function $c : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$. Given the state, the behavior of the BS to select the action is controlled by the scheduling policy $\pi : \mathcal{S} \to \mathcal{A}$. In time slot t, the DNN of DRL dynamically selects a suitable action $\boldsymbol{a}_t$ based on the present state $\boldsymbol{s}_t$ and the policy function, i.e., $\boldsymbol{a}_t = \pi(\boldsymbol{s}_t)$. As the return of the action, the agent receives the immediate reward $r_t = r(\boldsymbol{s}_t, \boldsymbol{a}_t)$ and immediate cost $c_t = c(\boldsymbol{s}_t, \boldsymbol{a}_t)$, then the system moves to the next state $\boldsymbol{s}_{t+1}$.

Let each time slot be a decision epoch. In each time slot, the state, action, reward and cost are defined as follows:

- **State:** The state of the CMDP is $\boldsymbol{s} = [s_u] \in \mathcal{S}$ which represents the average channel rates of eMBB and URLLC users.
- **Action:** Given the state $\boldsymbol{s} \in \mathcal{S}$, the action of the agent consists of the eMBB allocation factors and URLLC puncturing weights. In particular, the action $\boldsymbol{a} \in \mathcal{A}$ is a $1 \times 2U_{\text{eMBB}}$ vector and represented as $\boldsymbol{a} = [I_u^{\pi,\boldsymbol{s}}, w_u^{\pi,\boldsymbol{s}}], \forall u \in \mathcal{U}$ under the policy π.
- **Reward:** The reward function is defined as

$$r = r(\boldsymbol{s}, \boldsymbol{a}) = r_{\text{urllc}}^{\pi,\boldsymbol{s}} + r_{\text{eMBB}}^{\pi,\boldsymbol{s}}, \tag{15}$$

  where, $r_{\text{urllc}}^{\pi,\boldsymbol{s}}$ and $r_{\text{eMBB}}^{\pi,\boldsymbol{s}}$ are defined in (8) and (11), respectively.
- **Cost:** The cost function is related to the URLLC outage probability defined in **C4**. To transfer the statistical

Authorized licensed use limited to: University of Warwick. Downloaded on June 18,2024 at 10:22:54 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2024.3373722

constraint **C4** into a linear constraint $\overline{\boldsymbol{C4}}$, we apply the Markov's Inequality as [10]:

$$\overline{\boldsymbol{C4}}: \mathbb{P}\big[s_r < \frac{\sum_{m\in\mathcal{M}} U^{\pi,\boldsymbol{s}}_{\text{urllc},m}}{M}\big] \leq \frac{\mathbb{E}[U^{\pi,\boldsymbol{s}}_{\text{urllc},m}]}{s_r} \leq \zeta. \quad (16)$$

Thus, the cost function is defined as

$$c = c(\boldsymbol{s},\boldsymbol{a}) = \mathbb{E}[U^{\pi,\boldsymbol{s}}_{\text{urllc},m}]/s_r. \quad (17)$$

B. Problem Formulation of the CRN-DRL

The aim of the agent in the CRN-DRL model is to obtain an optimal policy that maximizes the expected discounted reward from the initial state without violating constraints. Let R_t and C_t denote the total discounted future return (also referred to as long-term rewards) and the total discounted future cost-return from time slot t onwards, which can be expressed as

$$R_t = r_t + \gamma r_{t+1} + \cdots = \sum\nolimits_{i=t}^{T} \gamma^{i-t} r_i, \quad (18)$$

and

$$C_t = c_t + \gamma c_{t+1} + \cdots = \sum\nolimits_{i=t}^{T} \gamma^{i-t} c_i, \quad (19)$$

where $\gamma \in [0,1]$ denotes the discount factor penalizing the rewards in the future and T represents the total number of iterations.

A policy π is safe if its expected cost-return remains below the constraint threshold of the reliability of URLLC users, thus the $\overline{\boldsymbol{C4}}$ could be maintained by letting [24]

$$\widetilde{\mathbf{C4}}: Q^{\pi}_{c}(\boldsymbol{s},\boldsymbol{a}) \leq \zeta, \quad (20)$$

where

$$Q^{\pi}_{c}(\boldsymbol{s},\boldsymbol{a}) = \mathbb{E}\big[C_1 | \boldsymbol{s}_1 = \boldsymbol{s}, \boldsymbol{a}_1 = \boldsymbol{a}; \pi\big]. \quad (21)$$

Over the episodes, the agent learns a safe policy π that maximizes the expectation of the long-term rewards from the start point $(\boldsymbol{s},\boldsymbol{a})$ under the policy π, i.e.,

$$Q^{\pi}_{r}(\boldsymbol{s},\boldsymbol{a}) = \mathbb{E}\big[R_1 | \boldsymbol{s}_1 = \boldsymbol{s}, \boldsymbol{a}_1 = \boldsymbol{a}; \pi\big]. \quad (22)$$

Let $J(\pi)$ and $J_{cost}(\pi)$ denote the expected discounted reward and cost, expressing as

$$J(\pi) = \int_{\mathcal{S}}\int_{\mathcal{A}} \mathbb{P}(\boldsymbol{a}|\boldsymbol{s}) Q^{\pi}_{r}(\boldsymbol{s},\boldsymbol{a}) d\boldsymbol{a} d\boldsymbol{s}, \quad (23)$$

$$J_{cost}(\pi) = \int_{\mathcal{S}}\int_{\mathcal{A}} \mathbb{P}(\boldsymbol{a}|\boldsymbol{s}) Q^{\pi}_{c}(\boldsymbol{s},\boldsymbol{a}) d\boldsymbol{a} d\boldsymbol{s}, \quad (24)$$

where $\mathbb{P}(\boldsymbol{a}|\boldsymbol{s})$ denotes the conditional probability of choosing action $\boldsymbol{a}$ given state $\boldsymbol{s}$. Formally, the optimization problem in (7) is rewritten as

$$\begin{aligned} \pi^* = \arg\max_{\pi\in\Pi} J(\pi) \quad (25)\\ \textbf{s.t.} \quad \mathbf{C1} \sim \mathbf{C3},\\ \widetilde{\mathbf{C4}}: J_{cost}(\pi) \leq \zeta, \end{aligned}$$

where the constraints **C1**-**C3** are defined the same as in (7).

C. Constraint-aware DRL algorithm

To address the statistical constraint $\widetilde{\mathbf{C4}}$, we employ the Lagrangian primal-dual method to incorporate the constraint into the long-term rewards function using Lagrange multiplier [24], [29]. This approach involves the formulation of the Lagrangian function, which can be expressed as

$$\mathcal{L}(\pi, \lambda_{cost}) = J(\pi) - \lambda_{cost}(J_{cost}(\pi) - \zeta), \quad (26)$$

where λ_{cost} is the multiplier of Lagrangian function. The point-wise maximization of Lagrangian with respect to policy π is called the dual function, can be expressed as follows:

$$\mathcal{D}(\lambda_{cost}) = \max_{\pi\in\Pi} \mathcal{L}(\pi, \lambda_{cost}), \quad (27)$$

The dual function in (27) serves as a lower bound on the Lagrangian's value in (26). A tighter low bound could narrow the gap between the policy derived from the dual function and the optimal solution of (25). Consequently, the dual problem, which aims to identify the tightest bound, is expressed as

$$\min_{\lambda_{cost}\geq 0} \mathcal{D}(\lambda_{cost}). \quad (28)$$

Then, the optimization problem in CRN-DRL is defined by solving

$$\begin{aligned} \min_{\lambda_{cost}\geq 0} \max_{\pi\in\Pi} J(\pi) - \lambda_{cost} J_{cost}(\pi) \quad (29)\\ \textbf{s.t.} \quad \mathbf{C1} \sim \mathbf{C3}. \end{aligned}$$

D. Actor-critic Learning Algorithm of CRN-DRL

For RL with continuous actions as in the eMBB/URLLC scheduling problem, deterministic policy gradient algorithms have been proposed as opposed to deep Q learning algorithms which are more appropriate for discrete actions. Our proposed CRN-DRL approach for addressing (29) is built upon the DDPG framework [30], which an off-policy actor-critic algorithm tailored for learning optimal policies in high-dimensional, continuous action spaces. This approach leverages key components from DQN, such as a replay buffer and slow-learning target networks, to enhance overall stability. The actor and critic networks, inspired by DPG, enable the algorithm to operate effectively over continuous action domains. However, different from the standard DDPG algorithm, the CRN-DRL algorithm incorporate two critic networks to update the weights of reward and cost DNNs. The learning structure of our proposed DRL algorithm is illustrated in Fig. 2. In the following, we explain the design of the essential elements of the proposed algorithm (i.e., the actor and critic networks) on continuous action domains, followed by the detailed learning process with replay buffer and the target networks.

1) Actor Networks: As shown in Fig. 2, the actor maintains an unconstrained policy function $\mu : \mathcal{S} \rightarrow \widetilde{\mathcal{A}}$ to map states to a specific unconstrained action $\widetilde{\boldsymbol{a}}$. The function μ is estimated by a DNN $\mu(\boldsymbol{s}|\theta)$, with trainable parameter θ. The actor network takes the current state observation as the input variable and generates a deterministic action as the output. The aim of the actor is to find θ of $\mu(\boldsymbol{s}|\theta)$ that maximizes (26) using a DRL-based approach. To balance the exploration and the exploitation dilemma, a stochastic noise should be added into

Authorized licensed use limited to: University of Warwick. Downloaded on June 18,2024 at 10:22:54 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2024.3373722

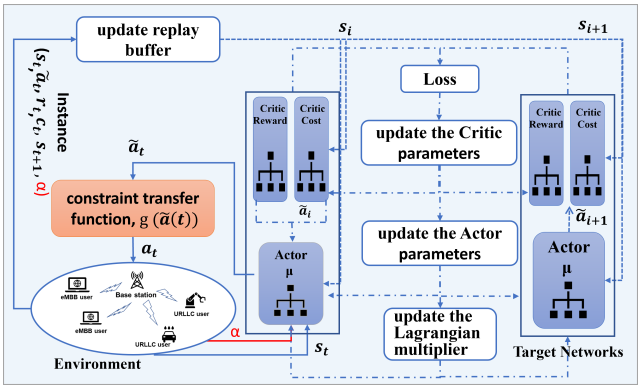


Fig. 2: The proposed architecture for the eMBB-URLLC joint system, the solid lines show how to generate the instance of the replay buffer in time slot t and the dash lines show how to update the parameters of the actor and critic networks. The risk-level (in red) is only considered in the CRS-DRL algorithm which will be introduced in Section V.

the actor network [31]. Thus, the action made by the actor network in time slot t is obtained as

$$\widetilde{\boldsymbol{a}}_t = \mu(\boldsymbol{s}_t|\theta) + N_{noise} \tag{30}$$

where N_{noise} denotes the stochastic noise which is included to ensure a broader exploration of the action space. The unconstrained DNN $\mu(\boldsymbol{s}|\theta)$ only ensures **C1** and **C2** in (29), i.e., $\widetilde{w}_u^{\mu,\boldsymbol{s}} \in [0,1]$ and $\widetilde{I}_u^{\mu,\boldsymbol{s}} \in [0,1]$. However, μ cannot guarantee $\sum_{u\in\mathcal{U}} \widetilde{I}_u^{\mu,\boldsymbol{s}} = 1, \forall \boldsymbol{s} \in \mathcal{S}$ (i.e., **C3** in (29)). To ensure the feasibility of the eMBB resource allocation, we propose a constraint transfer function $g(\cdot)$ to control the potential infeasible eMBB allocation factors. The transfer function takes each element in the output of $\mu(\boldsymbol{s}|\theta)$ as the input, and outputs the constrained action $\boldsymbol{a}$, i.e.,

$$\boldsymbol{a} = g(\widetilde{\boldsymbol{a}}) = [I_u^{\pi,\boldsymbol{s}}, w_u^{\pi,\boldsymbol{s}}], \forall u \in \mathcal{U}, \tag{31}$$

where $I_u^{\pi,\boldsymbol{s}} = \widetilde{I}_u^{\mu,\boldsymbol{s}} / \sum_{u'\in\mathcal{U}} \widetilde{I}_{u'}^{\mu,\boldsymbol{s}}$, and $w_u^{\pi,\boldsymbol{s}} = \widetilde{w}_u^{\mu,\boldsymbol{s}}$, $\forall u \in \mathcal{U}$. To obtain practical, implementable solutions, we normalized and rounded the eMBB allocation factors after the continuous optimization process.

Accordingly, in the proposed DRL algorithm, the scheduling policy function follows

$$\pi = \mu \circ g, \tag{32}$$

where $\circ$ denotes the composition of functions.

2) Critic Networks: In the CRN-DRL algorithm, the parameter π is updated by maximizing (26). However, $J(\pi)$ and $J_{cost}(\pi)$ are unknown. Thus, as shown in Fig.2, two different critic networks are used to estimate them.

Besides, based on (32), the reward and cost functions under policy π in (22) and (21) can be expressed as the functions under unconstrained policy function μ, i.e.,

$$\begin{aligned} Q_r^{\pi}(\boldsymbol{s},\boldsymbol{a}) &= \mathbb{E}\big[R_1|\boldsymbol{s}_1=\boldsymbol{s},\boldsymbol{a}_1=\boldsymbol{a};\pi\big] \\ &= \mathbb{E}\big[R_1|\boldsymbol{s}_1=\boldsymbol{s},\widetilde{\boldsymbol{a}}_1=\widetilde{\boldsymbol{a}};\mu\big] = Q_r^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}}) \end{aligned} \tag{33}$$

$$\begin{aligned} Q_c^{\pi}(\boldsymbol{s},\boldsymbol{a}) &= \mathbb{E}\big[C_1|\boldsymbol{s}_1=\boldsymbol{s},\boldsymbol{a}_1=\boldsymbol{a};\pi\big] \\ &= \mathbb{E}\big[C_1|\boldsymbol{s}_1=\boldsymbol{s},\widetilde{\boldsymbol{a}}_1=\widetilde{\boldsymbol{a}};\mu\big] = Q_c^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}}) \end{aligned} \tag{34}$$

Thus, we consider two differentiable DNN, parameterized by η and η_c, as the reward and cost critic networks to evaluate the performance of the actor's policy μ. Specifically, in time slot t, the critic networks output the reward function $Q_r^{\mu}(\boldsymbol{s}_t,\widetilde{\boldsymbol{a}}_t)$ and cost function $Q_c^{\mu}(\boldsymbol{s}_t,\widetilde{\boldsymbol{a}}_t)$ after taking action $\widetilde{\boldsymbol{a}}_t$, respectively, and state $\boldsymbol{s}_t$ as the input variables.

3) Replay Buffer and Mini-batch: The trainable parameters of actor and critic networks are learned through a large number of learning steps. The samples used to train the actor and critic networks are generated from exploring sequentially in the joint eMBB/URLLC scheduling environment. As the solid lines in Fig. 2 show, in each learning step, an experience tuple $(\boldsymbol{s}_t, \widetilde{\boldsymbol{a}}_t, r_t, c_t, \boldsymbol{s}_{t+1})$ is stored in the replay buffer $\mathcal{D}$. Specificity, in time slot t, the input of the learning plane is the state $\boldsymbol{s}_t$, which first goes through the actor network $\mu(\boldsymbol{s}_t|\theta)$ and produces a raw output $\widetilde{\boldsymbol{a}}_t$. After executing the transfer function for action $\boldsymbol{a}_t$, the reward r_t based on (15) the cost c_t based on (17) are obtained and the system moves to the next state $\boldsymbol{s}_{t+1}$. The replay buffer helps the agent to learn offline by gathering experience tuples accumulated so far rather than only the recent experiences. The oldest experience will be discarded when the replay buffer is full. As the dashed lines in Fig. 2 show, the critic and actor networks are updated by the mini-batch $\mathcal{X}$, i.e., the randomly selected X number of experience tuples from the replay buffer $\mathcal{D}$.

4) Learn Actor and Critic Parameters with Target Networks: The policy gradient method is applied to directly optimize the parameters of the actor and critic networks to maximize the objective function in (29). To estimate the reward and cost function under policy μ in each time slot, we can use the standard Bellman equation, i.e.,

$$Q_r^{\mu}(\boldsymbol{s}_t,\widetilde{\boldsymbol{a}}_t) = r_t + \gamma \sum_{\boldsymbol{s}_{t+1}\in\mathcal{S}} \mathbb{P}(\boldsymbol{s}_{t+1}|\boldsymbol{s}_t,\widetilde{\boldsymbol{a}}_t) Q_r^{\mu}(\boldsymbol{s}_{t+1},\widetilde{\boldsymbol{a}}_{t+1}), \tag{35}$$

$$Q_c^{\mu}(\boldsymbol{s}_t,\widetilde{\boldsymbol{a}}_t) = c_t + \gamma \sum_{\boldsymbol{s}_{t+1}\in\mathcal{S}} \mathbb{P}(\boldsymbol{s}_{t+1}|\boldsymbol{s}_t,\widetilde{\boldsymbol{a}}_t) Q_c^{\mu}(\boldsymbol{s}_{t+1},\widetilde{\boldsymbol{a}}_{t+1}), \tag{36}$$

where $\mathbb{P}(\boldsymbol{s}_{t+1}|\boldsymbol{s}_t,\widetilde{\boldsymbol{a}}_t)$ denotes the probability of transitioning from state $\boldsymbol{s}_t$ to state $\boldsymbol{s}_{t+1}$ by executing action $\widetilde{\boldsymbol{a}}_t$. However, if only using the single pair of actor and critic networks to iteratively learn and update the DNN parameters, (i.e., θ, η and η_c), the bellman equation in (35) and (36) will not converge. Thus, we consider that both actor and critic networks have their target networks, which help to stabilize the learning of action-value function and make it converge eventually. We define the target actor and critic networks as $\overline{\mu}(\boldsymbol{s}|\overline{\theta})$ parameterized by $\overline{\theta}$, $\overline{Q}_r^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}}|\overline{\eta})$ parameterized by $\overline{\eta}$, and $\overline{Q}_c^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}}|\overline{\eta_c})$ parameterized by $\overline{\eta_c}$, respectively. Therefore, with X numbers of selected experience from the replay buffer, the critic parameters can be updated by minimizing the loss function with stochastic gradient descent algorithm via η and η_c. The loss function is derived as

$$L_{n,r}^{c}(\eta) = \mathbb{E}_{\boldsymbol{s}_i,\widetilde{\boldsymbol{a}}_i\in\mathcal{X}}[(y_i^r - Q_r^{\mu}(\boldsymbol{s}_i,\widetilde{\boldsymbol{a}}_i|\eta))^2], \tag{37}$$

$$L_{n,c}^{c}(\eta_c) = \mathbb{E}_{\boldsymbol{s}_i,\widetilde{\boldsymbol{a}}_i\in\mathcal{X}}[(y_i^c - Q_c^{\mu}(\boldsymbol{s}_i,\widetilde{\boldsymbol{a}}_i|\eta_c))^2], \tag{38}$$

Authorized licensed use limited to: University of Warwick. Downloaded on June 18,2024 at 10:22:54 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2024.3373722

Algorithm 1 CRN-DRL algorithm for eMBB-URLLC scheduling.

1: Initialize replay buffer $\mathcal{D}$.
2: Initialize parameters (randomly) for actor and critic networks and the target networks.
3: **for** episodes=1 to K **do**
4: Initialize a random noise for action exploration.
5: Initialize the initial state s_1.
6: **for** each time slot t=1 to T **do**
7: Obtain raw action from μ with (30).
8: Apply constraint transfer function to obtain the feasible action $\boldsymbol{a}_t = g(\widetilde{\boldsymbol{a}}_t)$.
9: Execute $\boldsymbol{a}_t$ in emulator, get reward r_t, cost c_t and the next state $\boldsymbol{s}_{t+1}$.
10: Store experience $\left(\boldsymbol{s}_t, \widetilde{\boldsymbol{a}}_t, r_t, c_t, \boldsymbol{s}_{t+1}\right)$ in $\mathcal{D}$.
11: Sample a random mini-batch $\mathcal{X}$ from $\mathcal{D}$.
12: Update reward and cost critic networks by minimizing the loss in (37) and (38).
13: Update actor by the sampled policy gradient in (43).
14: Update parameters of the target actor and critic networks with (41), (42) and (44).
15: Update the lagrangian parameter λ_{cost} by the sampled policy gradient in (45).
16: **end for**
17: **end for**

where

$$y_i^r = r_i + \gamma \overline{Q}_r^{\mu}\left(\boldsymbol{s}_{i+1}, \overline{\mu}(\boldsymbol{s}_{i+1}|\overline{\theta})|\overline{\eta}\right). \tag{39}$$

$$y_i^c = c_i + \gamma \overline{Q}_c^{\mu}\left(\boldsymbol{s}_{i+1}, \overline{\mu}(\boldsymbol{s}_{i+1}|\overline{\theta})|\overline{\eta_c}\right). \tag{40}$$

The weights of the critic target network $\bar{\eta}$ and $\bar{\eta}_c$ is updated in a soft manner by

$$\bar{\eta} = \tau\eta + (1-\tau)\bar{\eta}, \tag{41}$$

$$\bar{\eta}_c = \tau\eta_c + (1-\tau)\bar{\eta}_c, \tag{42}$$

where $0 \leq \tau \ll 1$ is the soft updating rate.

While the actor adjusts the parameters θ of the deterministic policy μ by stochastic gradient ascent of (26), i.e.,

$$\begin{aligned}&\bigtriangledown_\theta \mathcal{L}(\pi, \lambda_{cost}) = \bigtriangledown_\theta \mathcal{L}(\mu, \lambda_{cost})\\ &= \mathbb{E}_{\boldsymbol{s}_i, \widetilde{\boldsymbol{a}}_i \in \mathcal{X}}(\bigtriangledown_{\mu(\boldsymbol{s}_i|\theta)} Q_r^{\mu}\left(\boldsymbol{s}_i, \mu(\boldsymbol{s}_i|\theta)\right)\\ &- \bigtriangledown_{\mu(\boldsymbol{s}_i|\theta)} \lambda_{cost} Q_c^{\mu}\left(\boldsymbol{s}_i, \mu(\boldsymbol{s}_i|\theta)\right)) \bigtriangledown_\theta \mu(\boldsymbol{s}_i|\theta),\end{aligned} \tag{43}$$

where the gradient $\bigtriangledown_{\mu(\boldsymbol{s}|\theta)} Q_r^{\mu}(\boldsymbol{s}, \mu(\boldsymbol{s}|\theta))|_{\boldsymbol{s}=\boldsymbol{s}_i}$ and $\bigtriangledown_{\mu(\boldsymbol{s}_i|\theta)} \lambda_{cost} Q_c^{\mu}\left(\boldsymbol{s}_i, \mu(\boldsymbol{s}_i|\theta)\right)$ are the policy evaluation from critic networks. The weights of the actor target network $\bar{\theta}$ is also updated in a soft manner by

$$\bar{\theta} = \tau\theta + (1-\tau)\bar{\theta}, \tag{44}$$

The weight λ_{cost} is learned by minimizing the loss of $J_{Lag}(\lambda_{cost})$:

$$J_{Lag}(\lambda_{cost}) = \mathbb{E}_{\boldsymbol{s}_i, \widetilde{\boldsymbol{a}}_i \in \mathcal{X}}\left(\lambda_{cost}\left(\zeta - Q_c^{\pi}(\boldsymbol{s}, \boldsymbol{a})\right)\right). \tag{45}$$

The dual problem (27) could be solved by updating the Lagrangian multipliers with sub-gradient descent as [24]

$$\lambda_{cost}(t+1) = \left[\lambda_{cost}(t) - \tau_{cost}\left(\sum Q_c^{\pi}(\boldsymbol{s}, \boldsymbol{a}) - \zeta\right)\right]^+ \tag{46}$$

where τ_{cost} is the step sizes and $[x]^+ = max(0, x)$.

5) Learning Process with Mini-batch: The details of learning algorithm of the proposed CRN-DRL are summarized in Algorithm 1. Steps 1 to 5 are the initialization of DRL. From Steps 7 to 10, the agent generates and stores the experience tuple of $\mathcal{D}$. Steps 11 to 14 explain how to train and update the parameters of the actor and critic networks in the CRN-DRL algorithm. Step 15 solves the (27) by updating the Lagrangian multipliers with sub-gradient descent.Our proposed algorithm optimizes the scheduling policy in the direction of the gradient of the actor and critic networks. Iteratively, the agent of the CRN-DRL explores the environment of the joint system better and better and learns the eMBB allocation factors and puncturing weights to achieve an optimal policy π^* eventually.

V. CRS-DRL-Based Joint eMBB-URLLC Scheduling Algorithm

The proposed CRN-DRL algorithm only concerns maximizing the expected total long-term rewards with constraint the expected cost smaller than the threshold. However, evolving URLLC mission-critical applications should enhance the capability to make robust and safe sequential scheduling decisions in complex joint eMBB-URLLC environments. In other words, the learning algorithm should capture the inherent randomness of the environment from the observable state to explicitly avoid the occurrences of catastrophic scheduling policies. In this case, optimizing a scheduling policy to maximize the expectation of the long-term rewards distribution is not ideal enough since the proposed CRN-DRL algorithm does not consider the possible risk of the stochastic future return, such as the high variance of eMBB utility, the heavy-tailed distribution of the not dropping URLLC packets. The agent is prone to choose a high variance policy (i.e., a high risk policy) with a high return in expectation rather than the low variance policies (i.e., low risk policies) with lower expected returns. Seeking a more robust joint scheduling policy of eMBB and URLLC traffic to reduce the likelihood of low total long-term rewards, in this section, we develop a CRS-DRL algorithm by considering the CVaR as the criterion to maximize. In this section, we present the definition of the risk criterion for the long-term rewards and costs functions, the proposed actor-critic architecture of the CRN-DRL based algorithm, and its learning process.

A. Problem Formulation with Risk criterion

To obtain a risk-sensitive policy, we start by modeling the distribution of stochastic future returns for all joint states and actions, therein the risk of various actions at each time slot can then be computed. Let R and C denote a random variable of the long-term rewards and costs, respectively. $P_R^{\pi}(R|\boldsymbol{s}, \boldsymbol{a})$ and $P_c^{\pi}(c|\boldsymbol{s}, \boldsymbol{a})$ denotes the probability distribution of R and C given the current state $\boldsymbol{s} \in \mathcal{S}$ and action $\boldsymbol{a} \in \mathcal{A}$ under the policy π. Fig. 3 compares the concept of the scheduling policies under the CRN-DRL and CRS-DRL algorithms. In the subfigures (a) and (b), the x-axis is R and y-axis is $P_R^{\pi}(R|\boldsymbol{s}, \boldsymbol{a})$. In (a), the blue line shows the expected value of future returns $Q_r^{\pi}(\boldsymbol{s}, \boldsymbol{a})$. The policy of the CRN-DRL algorithm is to optimize the average performance by changing the distribution

Authorized licensed use limited to: University of Warwick. Downloaded on June 18,2024 at 10:22:54 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2024.3373722

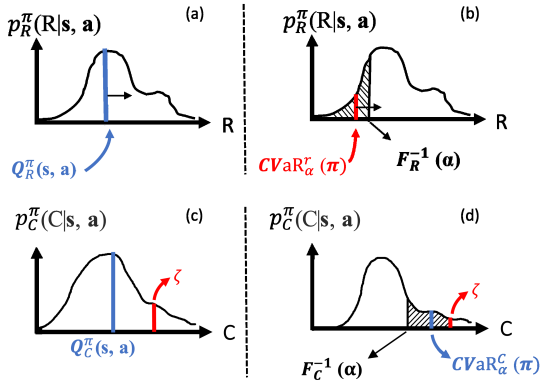


Fig. 3: **(a)**: CRN-DRL algorithm maximizes the expected long-term rewards $Q_R^\pi(\boldsymbol{s},\boldsymbol{a})$; **(b)**: CRS-DRL algorithm maximizes the risk criterion $CVaR_\alpha(\pi)$; **(c)**: CRN-DRL algorithm controls the expected long-term cost $Q_c^\pi(\boldsymbol{s},\boldsymbol{a})$ is smaller than the threshold; **(d)**: CRS-DRL algorithm reshapes the distribution of cost to ensure the risk criterion $CVaR_\alpha^c(\pi)$ smaller than the boundary ζ.

of R to shift the blue line to the right. However, the joint eMBB/URLLC system desires to averse the scheduling risk. Thus, the CRS-DRL in (b) intends to maximize the expected value of low performance cases to reduce the probabilities of undergoing very small future returns. In other words, we seek a scheduling policy which shifts the tail-end of the long-term rewards distribution $P_R^\pi(R|\boldsymbol{s},\boldsymbol{a})$ to the right, thus reducing the probability of experiencing low eMBB utility and large URLLC packets loss event. While, for the cost function, in Fig. 3 (c) and (d) the algorithm intends to reshape the $P_C^\pi(c|\boldsymbol{s},\boldsymbol{a})$ during the training process until the risk criterion for the cost (blue line) is shifted to the left side of the reliability boundary (red line).

To accomplish this idea, we define a positive scalar $\alpha \in [0,1]$ which determines the level of risk and the risk criterion is formally defined as the expected value of the BS experiencing the bottom α-percentile of the distribution over future returns. This criterions for long-term rewards and costs are defined as conditional value at risk and can be expressed as

$$\mathrm{CVaR}_\alpha^r(\pi) = \mathbb{E}_{P_R^\pi}[R|R < F_R^{-1}(\alpha)], \tag{47}$$

$$\mathrm{CVaR}_\alpha^c(\pi) = \mathbb{E}_{P_C^\pi}[C|C \geq F_C^{-1}(\alpha)], \tag{48}$$

where P_R^π and P_R^π denote the probability density function (PDF) of R and C respectively when following policy function π. F_R and F_C is the CDF of P_R^π and P_C^π, respectively. Therefore, the risk-averse long-term rewards and costs under risk level α becomes

$$J_\alpha^r(\pi) = \int_{\mathcal{S}}\int_{\mathcal{A}} \mathbb{P}(\boldsymbol{a}|\boldsymbol{s})\Gamma_r^\pi(\boldsymbol{s},\boldsymbol{a},\alpha)d\boldsymbol{a}d\boldsymbol{s}, \tag{49}$$

$$J_\alpha^c(\pi) = \int_{\mathcal{S}}\int_{\mathcal{A}} \mathbb{P}(\boldsymbol{a}|\boldsymbol{s})\Gamma_c^\pi(\boldsymbol{s},\boldsymbol{a},\alpha)d\boldsymbol{a}d\boldsymbol{s}, \tag{50}$$

where $\Gamma_r^\pi(\boldsymbol{s},\boldsymbol{a},\alpha) \doteq \mathrm{CVaR}_\alpha^r(\pi)$ and $\Gamma_c^\pi(\boldsymbol{s},\boldsymbol{a},\alpha) \doteq \mathrm{CVaR}_\alpha^c(\pi)$ denotes the closed-form of the risk criterion in (47) and (48), respectively. The closed-form of the risk criterion in (49) and (50) could be obtained by substituting the mean and variance of the long-term rewards and costs distribution into (47) and (48), the detail will be explained in the next subsection. For a certain risk-level α, we optimize the policy until $\Gamma_c^\pi(\boldsymbol{s},\boldsymbol{a},\alpha)$ satisfied the URLLC constraint

$$\Gamma_c^\pi(\boldsymbol{s},\boldsymbol{a},\alpha) \leq \zeta \tag{51}$$

With using the Lagrangian multiplier λ_{cost}, the optimization problem under risk-level α in CRS-DRL is defined by solving

$$\min_{\lambda_{cost}\geq 0}\max_{\pi\in\Pi} J_\alpha^r(\pi) - \lambda_{cost}J_\alpha^c(\pi) \tag{52}$$

$$\textbf{s.t.} \qquad \textbf{C1} \sim \textbf{C3}.$$

By maximizing the objective function, the agent reduces the variance of long-term eMBB rewards. Thereby, minimizing the risk on eMBB transmissions and enhancing its throughput. At the meantime, the CRS-DRL algorithm effectively captures and improves the risk tail distribution of the long-term URLLC rewards (i.e., the probability of not dropping URLLC packets), and control the risk-tail of the cost function, thereby, ensuring a robust high URLLC reliability. The learned policy could act differently from the same state depending on α. As $\alpha \to 0$, this risk-sensitive criterion evaluates the policy with focus on the "*worse cases*" scenarios. Conversely, a larger value of α approaches less risk-sensitive behavior and $\alpha \to 1$ leads to the risk-neutral control based model.

B. Actor-critic Learning Algorithm of CRS-DRL

To solve the optimization problem (52), the proposed actor-critic architecture of the CRS-DRL algorithm (as shown in Fig. 2) has two modifications compared to the risk-neutral one. Firstly, an additional input α is added to both the actor and the critic networks to reduce the scheduling variance. Secondly, the distributional critic networks are considered to replace the standard critic networks in the risk-neutral, which are modified to predict the full distribution over future reward and cost instead of simply the expectation. The reason to use a distributional critic is that computing the CVaR measure directly for long time horizons is expensive [32], but as an alternative, the closed-form of the risk criteria (i.e., $\mathrm{CVaR}_\alpha^r(\pi)$, and $\mathrm{CVaR}_\alpha^c(\pi)$) can be derived as a function of the output parameters of the distributional critic network in each time slot.

In the following sub-sections, we will begin by elucidating the derivation of the closed-form risk criteria. These criteria are expressed as functions of the mean and variance of the long-term rewards or costs distribution and are dependent on the state and action at each learning step. Subsequently, we will delve into how the proposed distributional critic networks govern the distribution of long-term returns and costs, leveraging the mean and variance. Following this, we will outline the details of how to update the weights of the actor and critic networks in each learning step. Finally, we will provide an overview of the learning process for the CRS-DRL algorithm.

1) Closed-form of CVaR: Let us define $R^\pi(\boldsymbol{s},\boldsymbol{a})$ as the long-term rewards from starting point $(\boldsymbol{s},\boldsymbol{a})$ under the policy π until reaching a terminal state. To derive the closed-form of risk criterion for the robust scheduling policy, we assume the

Authorized licensed use limited to: University of Warwick. Downloaded on June 18,2024 at 10:22:54 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2024.3373722

long-term rewards has a Gaussian distribution as [33], [34], i.e.,

$$R^{\pi}(\boldsymbol{s},\boldsymbol{a}) \sim \mathcal{N}\left(Q_r^{\pi}(\boldsymbol{s},\boldsymbol{a}), \Upsilon_r^{\pi}(\boldsymbol{s},\boldsymbol{a})\right), \tag{53}$$

where $Q_r^{\pi}(\boldsymbol{s},\boldsymbol{a})$ and $\Upsilon_r^{\pi}(\boldsymbol{s},\boldsymbol{a})$ denote the mean and variance of the long-term rewards, respectively. Defining the mean as

$$Q_r^{\pi}(\boldsymbol{s},\boldsymbol{a}) = \mathbb{E}_{p_R^{\pi}}[R|\boldsymbol{s},\boldsymbol{a}], \tag{54}$$

and the variance as

$$\Upsilon_r^{\pi}(\boldsymbol{s},\boldsymbol{a}) = \mathbb{E}_{p_R^{\pi}}[R^2|\boldsymbol{s},\boldsymbol{a}] - (Q_r^{\pi}(\boldsymbol{s},\boldsymbol{a}))^2. \tag{55}$$

Then, the closed-form of the risk criterion in (49) is obtained by substituting the mean and variance functions (54) and (55) into (47) as [34]

$$\begin{aligned}\Gamma_r^{\pi}(\boldsymbol{s},\boldsymbol{a},\alpha) &= Q_r^{\pi}(\boldsymbol{s},\boldsymbol{a}) - (\phi(\alpha)/\Phi(\alpha))\sqrt{\Upsilon_r^{\pi}(\boldsymbol{s},\boldsymbol{a})}, \\ &= Q_r^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}}) - (\phi(\alpha)/\Phi(\alpha))\sqrt{\Upsilon_r^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}})},\end{aligned} \tag{56}$$

where $\phi(\cdot)$ is the standard normal distribution and $\Phi(\cdot)$ is its CDF, i.e., $\Phi(\cdot) = \frac{1}{2}(1 + \text{erf}(x/\sqrt{2}))$.

The same process can be applied to the cost function. The closed-form of the risk criterion $\Gamma_c^{\pi}(\boldsymbol{s},\boldsymbol{a},\alpha)$ in (50) is obtained by substituting the mean $Q_c^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}})$ and variance $\Upsilon_c^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}})$ of cost function into (48) as [34]

$$\Gamma_c^{\pi}(\boldsymbol{s},\boldsymbol{a},\alpha) = Q_c^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}}) - (\phi(\alpha)/\Phi(\alpha))\sqrt{\Upsilon_c^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}})}. \tag{57}$$

2) Distributional Critic: In each time slot, the $\Gamma_r^{\pi}(\boldsymbol{s},\boldsymbol{a},\alpha)$ can be computed from the learned distribution parameters of critic networks. The distributional critic is a differentiable DNN, $f_{critic,r}^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}}|\kappa)$, parameterized by κ, which is also used to evaluate the performance of the actor's policy μ. The distributional critic in the CRS-DRL takes the states, raw actions and risk-level as the inputs. Different from risk-neutral case, instead of producing a scalar value for approximating $Q_r^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}})$, the critic network governs the distribution of future returns and outputs the mean and variance of the long-term rewards distribution, i.e.,

$$f_{critic,r}^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}}|\kappa) \rightarrow \{Q_r^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}}), \Upsilon_r^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}})\}. \tag{58}$$

3) Learn Actor and Critic Parameters with Target Networks: To learn the distributional critic in CRS-DRL, the distributional Bellman operator of mean long-term rewards can be iteratively updated under actor policy μ by the standard Bellman equation as (35). The projection equation for estimating the variance of long-term rewards is expressed as [34]

$$\begin{aligned}\Upsilon_r^{\mu}(\boldsymbol{s}_t,\widetilde{\boldsymbol{a}}_t) &= r_t^2 + \gamma^2 \sum_{\boldsymbol{s}_{t+1}\in\mathcal{S}} \mathbb{P}(\boldsymbol{s}_{t+1}|\boldsymbol{s}_t,\widetilde{\boldsymbol{a}}_t)\Upsilon_r^{\mu}(\boldsymbol{s}_{t+1},\widetilde{\boldsymbol{a}}_{t+1}) \\ &+ \gamma^2 \sum_{\boldsymbol{s}_{t+1}\in\mathcal{S}} \mathbb{P}(\boldsymbol{s}_{t+1}|\boldsymbol{s}_t,\widetilde{\boldsymbol{a}}_t) Q_r^{\mu}(\boldsymbol{s}_{t+1},\widetilde{\boldsymbol{a}}_{t+1})^2 - Q_r^{\mu}(\boldsymbol{s}_t,\widetilde{\boldsymbol{a}}_t)^2 \\ &+ 2\gamma r_t \sum_{\boldsymbol{s}_{t+1}\in\mathcal{S}} \mathbb{P}(\boldsymbol{s}_{t+1}|\boldsymbol{s}_t,\widetilde{\boldsymbol{a}}_t) Q_r^{\mu}(\boldsymbol{s}_{t+1},\widetilde{\boldsymbol{a}}_{t+1}).\end{aligned} \tag{59}$$

The target networks are used for the actor and critic networks in CRS-DRL in order to improve the stability. The parameter of the critic (i.e., κ) can be learned by the p-Wasserstein distance algorithm [35], [36], which defines the distance between two distributions as

$$W_p(u,v) \doteq \left(\int_0^1 |F_u^{-1}(s) - F_v^{-1}(s)|^p ds\right)^{1/p}, \tag{60}$$

where $u \sim \mathcal{N}(\overline{Q}_r^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}}|\bar{\kappa}), \overline{\Upsilon}_r^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}}|\bar{\kappa}))$ is the target critic network, $v \sim \mathcal{N}(Q_r^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}}|\kappa), \Upsilon_r^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}}|\kappa))$ is the estimated critic network. F_u and F_v are the corresponding CDFs, respectively. In this paper, we use the 2-Wasserstein distance [34] based on the projection equations in (35) and (59) to update the parameters in the critic networks. Specifically, with the set of mini-batch denoted as $\mathcal{X}$, we minimize the critic loss function, which is expressed as

$$\begin{aligned}L_{r,r}^{c}(\kappa) = \mathbb{E}_{\boldsymbol{s},\widetilde{\boldsymbol{a}}\in\mathcal{X}}\Big[&(y_i^r - Q_r^{\mu}(\boldsymbol{s}_i,\widetilde{\boldsymbol{a}}_i|\kappa))^2 \\ &+ \left(z_i^r + \Upsilon_r^{\mu}(\boldsymbol{s}_i,\widetilde{\boldsymbol{a}}_i|\kappa) - 2(z_i^r \Upsilon_r^{\mu}(\boldsymbol{s}_i,\widetilde{\boldsymbol{a}}_i|\kappa))^{\frac{1}{2}}\right)\Big],\end{aligned} \tag{61}$$

where

$$y_i^r = r_i + \gamma \overline{Q}_r^{\mu}(\boldsymbol{s}_{i+1}, \overline{\mu}(\boldsymbol{s}_{i+1}|\bar{\theta})|\bar{\kappa}), \tag{62}$$

$$\begin{aligned}z_i^r = {} & r_i^2 + \gamma^2 \overline{\Upsilon}_r^{\mu}(\boldsymbol{s}_{i+1}, \overline{\mu}(\boldsymbol{s}_{i+1}|\bar{\theta})|\bar{\kappa}) - Q_r^{\mu}(\boldsymbol{s}_i,\widetilde{\boldsymbol{a}}_i|\kappa)^2 \\ & + \gamma^2 \overline{Q}_r^{\mu}(\boldsymbol{s}_{i+1}, \overline{\mu}(\boldsymbol{s}_{i+1}|\bar{\theta})|\bar{\kappa})^2 + 2\gamma r_i \overline{Q}_r^{\mu}(\boldsymbol{s}_{i+1}, \overline{\mu}(\boldsymbol{s}_{i+1}|\bar{\theta})|\bar{\kappa}).\end{aligned} \tag{63}$$

The critic in CRS-DRL minimizes the 2-Wasserstein distance by backpropagating the gradient $\frac{L_{r,r}^{c}(\kappa)}{\partial\kappa}$. Then we can use the critic nework to estimate $Q_r^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}})$ and $\Upsilon_r^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}})$ in (58) and then compute the CVaR_{α}^{r} measure (i.e., $\Gamma_r^{\pi}(\boldsymbol{s},\boldsymbol{a},\alpha)$) in closed-form (56) with $Q_r^{\pi}(\boldsymbol{s},\boldsymbol{a}) = Q_r^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}})$ and $\Upsilon_r^{\pi}(\boldsymbol{s},\boldsymbol{a}) = \Upsilon_r^{\mu}(\boldsymbol{s},\widetilde{\boldsymbol{a}})$. The loss function for the cost critic network $L_{r,c}^{c}(\kappa_c)$, parametered by κ_c, is derived by the same procedure, from (58) to (61).

Finally, the actor weights can be trained by backpropagating the gradient back through the actor network $\mathcal{L}_r(\pi,\lambda_{cost}^r)$, which is defined as

$$\mathcal{L}_r(\pi,\lambda_{cost}^r) = \Gamma_r^{\pi}(\boldsymbol{s},\boldsymbol{a},\alpha) - \lambda_{cost}^r(\Gamma_c^{\pi}(\boldsymbol{s},\boldsymbol{a},\alpha) - \zeta), \tag{64}$$

To satisfy the URLLC reliability constraint is less than the threshold ζ, the Lagrangian weight λ_{cost}^r can be learned by minimizing the loss of $J_{Lag,r}(\lambda_{cost}^r)$

$$J_{Lag,r}(\lambda_{cost}^r) = \mathbb{E}_{\boldsymbol{s}_i,\widetilde{\boldsymbol{a}}_i\in\mathcal{X}}\left(\lambda_{cost}^r\left(\zeta - \Gamma_c^{\pi}(\boldsymbol{s},\boldsymbol{a},\alpha)\right)\right). \tag{65}$$

4) Learning Process with Mini-batch: Algorithm 2 provides the illustration of the learning process of the proposed CRS-DRL algorithm. During training, a different input α will have a different loss function $\mathcal{L}_r(\pi,\lambda_{cost}^r)$. To train for all α, we uniformly sample $\alpha \sim Uniform(0,1)$ during the start of an episode and fix α for the entirety of that episode. Steps 1 to 5 are related to the initialization of DRL. From Steps 7 to 10, the agent generates and stores the experience tuple of $\mathcal{D}$. Steps 11 to 14 explain how to train and update the parameters of the actor and critic networks in the CRS-DRL algorithm. Step 15 enures that the URLLC reliability constraint is met and remains below the threshold ζ, the Lagrangian weight λ_{cost}^r can be learned by minimizing the loss $J_{Lag,r}$.

Authorized licensed use limited to: University of Warwick. Downloaded on June 18,2024 at 10:22:54 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2024.3373722

Algorithm 2 CRS-DRL algorithm for eMBB-URLLC scheduling.

Initialize replay buffer $\mathcal{D}$;
Initialize parameters (randomly) for actor and critic networks and the target networks;
for episodes=1 to K **do**
  Initialize a random noise for action exploration.
  Initialize the initial state s_1 and $\alpha \sim U(0,1)$.
  for each time slot t=1 to T **do**
    Obtain raw action from μ with (30).
    Apply constraint transfer function to obtain the feasible action $\boldsymbol{a}_t = g(\widetilde{\boldsymbol{a}}_t)$.
    Execute $\boldsymbol{a}_t$ in emulator and get reward r_t, cost c_t and the next state $\boldsymbol{s}_{t+1}$.
    Store transition $\left(\boldsymbol{s}_t, \widetilde{\boldsymbol{a}}_t, r_t, c_t, \boldsymbol{s}_{t+1}, \alpha\right)$ in $\mathcal{D}$.
    Sample a random mini-batch $\mathcal{X}$ from $\mathcal{D}$.
    Update critic by minimizing the loss functions $L^c_{r,r}(\kappa)$ and $L^c_{r,c}(\kappa)$.
    Update actor by the sampled policy gradient with the CVaR objective in (64).
    Update the parameters of the target actor and critic networks with (41), (42) and (44).
    Update the Lagrangian parameter λ^r_{cost} by the sampled policy gradient in (65).
  end for
end for

VI. Results

A. Experimental Settings

We evaluated the effectiveness of the proposed CRS-DRL algorithm through simulations, compared with the CRN-DRL algorithms and a DRL model introduced in [20], which lacks the queuing mechanism, without constraining the statistical outage of URLLC, and ignores risk-sensitive features. Our objective is to highlight the robustness of the CRS-DRL approach.

We design an OpenAI gym as the dynamic environment provided with the given state space as defined in Section IV-A. A five-layer, fully connected feedforward DNN is utilized to construct the neural network for actor μ in both proposed CRN-DRL and CRS-DRL algorithms. The inputs of the actor and its target network is the instance vector of eMBB channel rates $\boldsymbol{s}$ of size U_{embb}. The risk level is also considered as an input variable for the CRS-DRL algorithm. The three hidden layers of the actor neural network include 128, 256, and 128 neurons, correspondingly. The rectified linear unit (ReLU) is used as the activation function for the hidden layers. Sigmoid is used as the activation function for the output layer of the actor network to scale the puncturing weights and eMBB allocation factors between zero and one. To ensure a broader exploration of the action space, we add a small random noise to $\widetilde{\boldsymbol{a}}_t$ which is assumed to have the Gaussian distribution with zero mean and a standard deviation of 0.02. Then, all entries in $\widetilde{\boldsymbol{a}}_t$ are clipped to ensure they are within $[0,1]$ and the eMBB allocation factors will be normalized by the constraint transfer function afterward. The reward and cost critic networks of the CRN-DRL is a five-layer feedforward neural network with the same intermediate layer structure and activation functions as the actor μ. The inputs of the critic and the target networks is the instance rate vector $\boldsymbol{s}$ and the raw output $\widetilde{\boldsymbol{a}}$ of μ. The CRS-DRL has a similar structure but with the risk level as an additional input. Linear activation function is used for the output layer of the CRN-DRL critic network to obtain the action-value function. *softplus* function is used for the CRS-DRL to guarantee the estimated the variance of the critic networks will always be positive.

TABLE II: System Configuration

Parameters	Values
The discount factor	$\gamma = 0.99$
The step size for the target networks	$\tau = 0.001$
The step size for Lagrangian parameter	τ_c=0.1
The initial Lagrangian multiplier	λ_{cost}=0.01
URLLC outage threshold	ζ=0.3
The number of channels	B=10
The set of possible channel rates for eMBB users	$\mathcal{C}_{\text{embb}} = \{0.1, 0.5, 1\}$ Mbps
The set of possible channel rates for URLLC users	$\mathcal{C}_{\text{uRLLC}} = \{3, 5, 7\}$ packets/mini-slot
The set of Poisson URLLC packet arrival rate	$\lambda \in \{0.1, 0.2, \ldots, 1, 2\}$ packets/mini-slot

Off-policy training is performed with an experience replay buffer of up to $d = 10^5$ $\{\boldsymbol{s}_i, \widetilde{\boldsymbol{a}}_i, r_i, c_i, \boldsymbol{s}_{i+1}\}$ tuples. The number of random samples in mini-batches used for training is 128. The Adam optimizer is used with a learning rate of 10^{-4}. We consider a system with three eMBB users with the setting in Table II. All simulation results are generated by using Tensorflow 1.13.0 on the Python 3.6 platform. s_r is expressed in packets/mini-slot in this paper. This unit of measurement aligns cohesively with the assumption of URLLC packets following a Poisson distribution, where the average arrival rate is specified in packets/mini-slot.

B. Algorithm Complexity

The time complexity of the proposed algorithm depends on the complexity of the neural network operations involved, including the number of layers, the size of the nodes, the length of episodes (K and T). Based on the setting in Section VI.A, for the actor network with five layers, the number of nodes of each layer is denoted by i_a, j_a, k_a, l_a, q_a, respectively. Thus the actor has four matrices W_{j_a,i_a}, W_{k_a,j_a}, W_{l_a,k_a} and W_{q_a,l_a} representing weights between these layers. For instance, W_{j_a,i_a} contains the weights going from layer i_a to layer j_a. For propagating from layer i_a to j_a, the time complexity has an order of $O(j_a i_a TK)$. Moreover, the computational time for applying the activation function has complexity of $\mathcal{O}(j_a TK)$ since it is an element-wise operation. Therefore, from layer i_a to j_a, the complexity is an order of $\mathcal{O}(j_a i_a TK + j_a TK) = \mathcal{O}(j_a TK(i_a+1)) = \mathcal{O}(j_a i_a TK)$. The total time complexity for the actor is $\mathcal{O}(j_a i_a TK + k_a j_a TK + l_a k_a TK + q_a l_a TK) = \mathcal{O}(TK(i_a j_a + j_a k_a + l_a k_a + q_a l_a))$. For each critic network, we have five layers, with the number of nodes are i_c, j_c, k_c, l_c, and q_c, thus the total time complexity for critic networks is $\mathcal{O}(TK(i_c j_c + j_c k_c + l_c k_c + q_c l_c))$. The time complexity of the proposed algorithm is $\mathcal{O}(TK(i_a j_a + j_a k_a + l_a k_a + q_a l_a + i_c j_c + j_c k_c + l_c k_c + q_c l_c))$.

C. Convergence of Learning

During training, we use the average episodic return as the metric. In Fig. 4(a) and Fig. 4(b), we depict the learning convergence of the proposed CRN-DRL and CRS-DRL algorithms over 10^6 training steps for different URLLC arrival

Authorized licensed use limited to: University of Warwick. Downloaded on June 18,2024 at 10:22:54 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2024.3373722

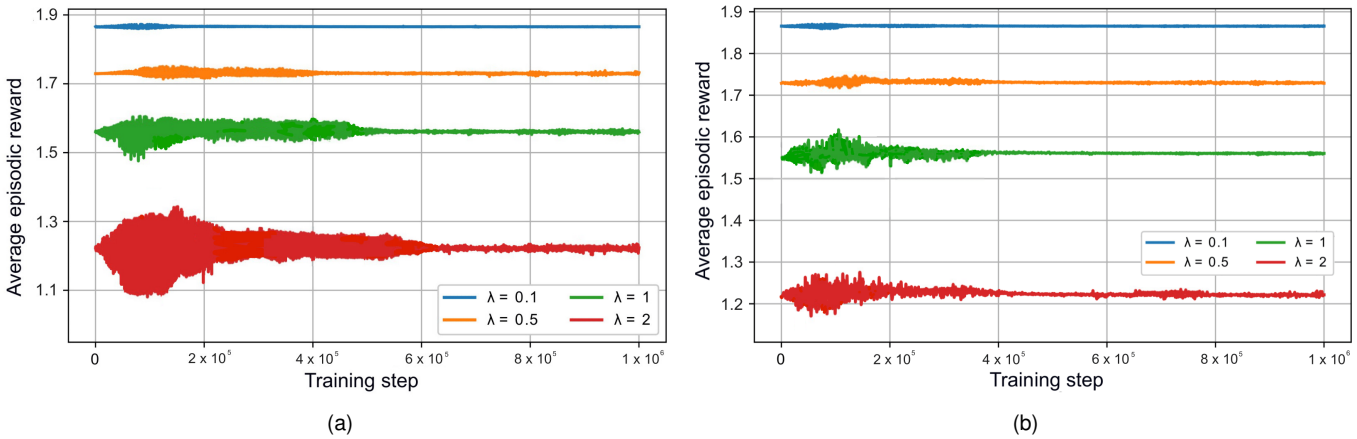


Fig. 4: Learning convergence of proposed algorithms for different URLLC packet arrival rates λ; (a) CRN-DRL (b) CRS-DRL with $\alpha = 0.0$.

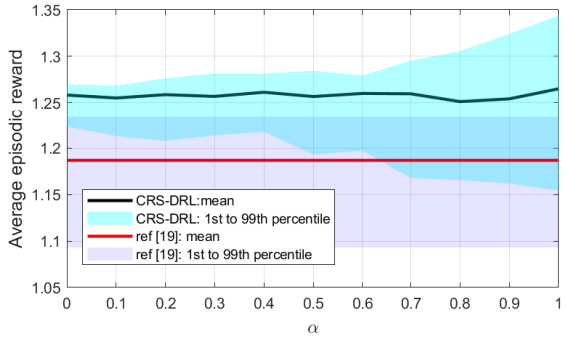


Fig. 5: The average episodic reward and their 1st and 99th percentile as a function of α, $\lambda = 2$ packets/mini-slot.

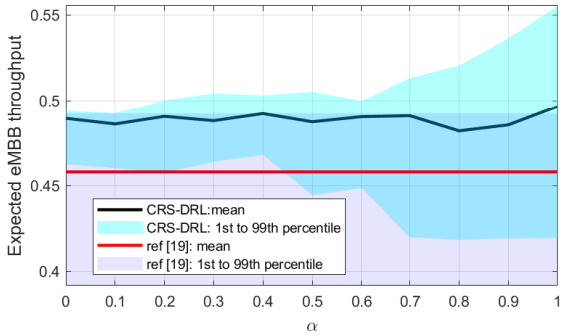


Fig. 6: The average total normalized eMBB throughput and their 1st and 99th percentiles versus α for $\lambda = 2$ packets/mini-slot.

rates, respectively. Each point on a curve represents the reward value averaged over the past 200 time slots. In general, the proposed CRN-DRL algorithm converges smoothly within 6×10^5 training steps, while CRS-DRL with $\alpha = 0.0$ converges more quickly within 4×10^5 training steps. Furthermore, the average episodic return of CRN-DRL fluctuates more compared to CRS-DRL. This discrepancy arises because CRS-DRL aims to minimize the variance of the long-term rewards distribution during training, whereas CRN-DRL only optimizes the mean value of long-term rewards.

D. Risk-sensitivity Evaluations

After training, we use 800 test runs to validated the proposed CRN-DRL (α=1.0) and CRS-DRL algorithms through simulations, comparing it with the proposed RL-based algorithm in [20], which has a actor-critic architecture, but lacks a queuing mechanism, a statistical reliability constraint, and risk considerations.

Figures 5-7 show the average episodic rewards, average total normalized eMBB throughput and probability of not dropping URLLC packets with $\lambda = 2$ packets/mini-slot. Based on the average episodic rewards in Fig. 5, the CRN-DRL algorithm has a higher likelihood of experiencing unsatisfactorily low instantaneous rewards. This is evidenced by the wider shaded blue area at α=1.0, which indicates the 1st and 99th percentiles

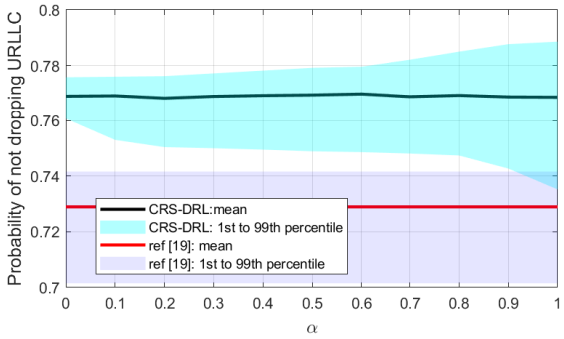


Fig. 7: The average probability of not dropping URLLC packets and their 1st and 99th percentiles versus α for $\lambda = 2$ packets/mini-slot.

of rewards. The wider region of 1st and 99th percentiles of rewards also obtained for the benchmark algorithm in [20]. Thus, the results show that the risk-neutral policies are not aware of the potential long tail of the long-term rewards which is reflected in low values of total normalized eMBB throughput (as shown in Fig. 6) and the probabilities of not dropping URLLC packets (as shown in Fig. 7).

As a comparison, the CRS-DRL policies keep the variance of the long-term rewards distribution in a safe range, which reduce the region of the 1st and 99th percentiles of rewards.

Authorized licensed use limited to: University of Warwick. Downloaded on June 18,2024 at 10:22:54 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2024.3373722

As shown in Fig. 5, when α decreases, the policy under CRS-DRL becomes risk-averse and more robust, especially when α is in range of $\{0.0, 0.5\}$. The same trend could be observed in Fig. 6, and Fig. 7 for the average total normalized eMBB throughput and the probability of not dropping URLLC packets, respectively. Notably, the proposed CRN-DRL and CRS-DRL policies achieve similar performance in terms of average rewards. In contrast, the RL-based algorithm in ref [20] yields lower expected rewards. This discrepancy is due to the lack of consideration for queue system for URLLC packets, resulting in a higher count of URLLC drops. Consequently, base stations need to allocate more channels for URLLC puncturing on eMBB users, which leads to a lower eMBB throughput.

We also evaluated the variance of the long-term rewards distribution observed during the test stage when assessing the performance of the learning algorithms under different risk-level settings. Table. III shows how the estimated variance of long-term rewards changes as the risk level α is adjusted, where the variance in high risk case ($\alpha = 0.0$) is 62.33% and 65.24% compared to the risk-neutral case for the CRN-DRL and [20], respectively. Besides, Table. IV shows that the estimated variance increases under all the learning algorithms when the URLLC arrival rate increases. However, the increase rate is slower in the case of CRS-DRL and the gaps between the risk-neutral algorithms and the CRS-DRL algorithm get larger as λ increases.

TABLE III: Estimated mean reward Variances under different risk-level α for the proposed algorithms, $\lambda = 2$ packets/mini-slot

α	0	0.1	0.2	0.3	0.4	0.5
Variance	0.081	0.088	0.095	0.102	0.109	0.116
α	0.6	0.7	0.8	0.9	CRN-DRL	Ref [20]
Variance	0.127	0.138	0.157	0.182	0.215	0.223

TABLE IV: Estimated mean Variances for CRN-DRL and CRS-DRL with $\alpha = 0$ under different URLLC arrival rates λ

λ	0.1	0.2	0.3	0.4
CRS-DRL, $\alpha = 0$	0.0014	0.0028	0.0049	0.0039
CRN-DRL	0.0066	0.0128	0.0144	0.0213
Ref [20]	0.0070	0.0135	0.0150	0.0225
λ	0.5	0.6	0.7	1
CRS-DRL, $\alpha = 0$	0.0047	0.0057	0.0039	0.0113
CRN-DRL	0.0228	0.0248	0.0304	0.0370
Ref [20]	0.0233	0.0249	0.0335	0.0392

E. Estimated long-term rewards distribution

In Fig. 8 and Fig. 9, we conducted 800 test runs to assess the empirical PDF and CDF of average episodic rewards under the proposed CRN-DRL and CRS-DRL algorithms, with λ set at 2 packets/mini-slot. Notably, as depicted in Fig. 8, we observed that for the average episodic rewards distributions at risk levels of α=0.0 and α=0.2, the variances are smaller when compared to the case of CRN-DRL at α=1.0, resulting in PDFs with heavy tails. Similar insights can be obtained from the results presented in Fig. 9, highlighting how the proposed CRS-DRL algorithm enhances the tail of the distribution. For instance, the risk-neutral algorithm exhibits a larger CDF when the average episodic rewards are smaller than the mean value. These findings substantiate the effectiveness of CRS-DRL.

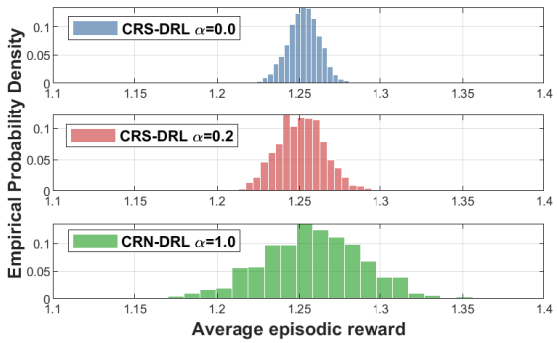


Fig. 8: Empirical PDFs of 800 test runs of long-term returns under the CRS-DRL and CRN-DRL algorithms, $\lambda = 2$ packets/mini-slot

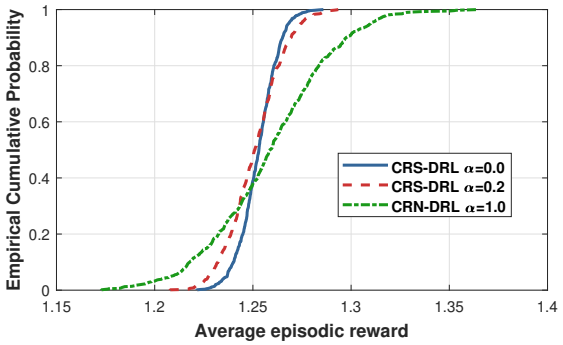


Fig. 9: Empirical CDFs of returns observed in validation of the DRL policy by optimizing expectation (i.e., CRN-DRL) and CVaR (i.e., CRS-DRL), $\lambda = 2$ packets/mini-slot.

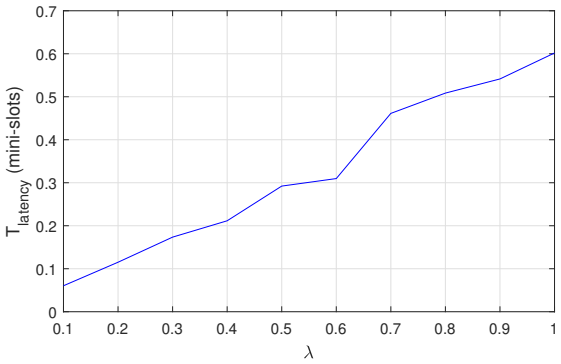


Fig. 10: Average delay of served URLLC packets versus URLLC arrival rate.

F. Evaluate of URLLC Latency

To evaluate the effectiveness of the proposed URLLC queue system, we defined URLLC latency as the mean queueing time for URLLC served packets i.e.,

$$T_{latency} = (\mathbb{E}[Q_{m-1}] - \mathbb{E}[D_m])/\mathbb{E}[\mu_m]. \quad (66)$$

In Fig. 10, we use the optimized puncturing weights from 800 test runs to numerically plotted the URLLC latency against the URLLC arrival rate λ, when applying different learning algorithm the CRS-DRL and RL-based approach in [20]. A larger value of $T_{latency}$ suggests that the system can transmit more URLLC packets with queuing at the base station, whereas a smaller $T_{latency}$ implies that URLLC packets

Authorized licensed use limited to: University of Warwick. Downloaded on June 18,2024 at 10:22:54 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2024.3373722

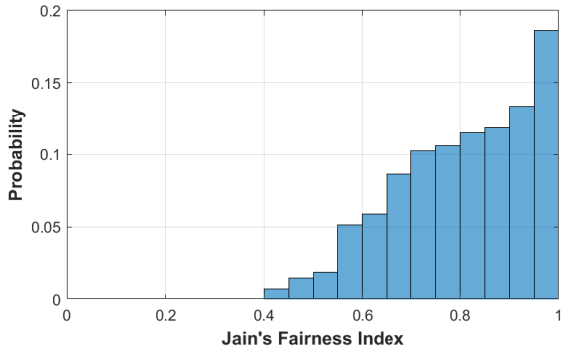


Fig. 11: The fairness index distribution for 800 test runs under the CRS-DRL algorithm with risk-level $\alpha = 0.0$.

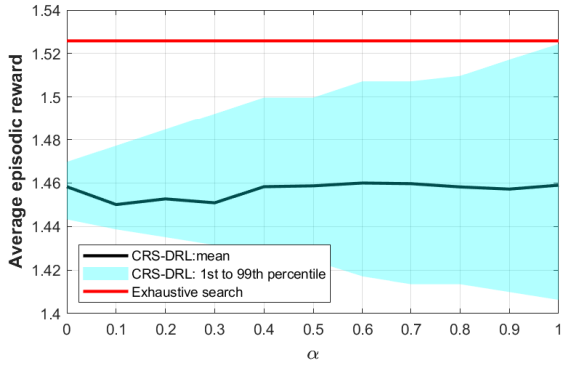


Fig. 12: The average episodic reward and their 1st and 99th percentile as a function of α, λ = 0.5 packets/mini-slot, under the proposed CRS-DRL algorithm and the exhaustive search.

can be transmitted directly without queuing. The results in Fig. 10 shows when the number of URLLC arrival packets increases, the system must queue more URLLC packets to meet the low-latency requirement.

G. Evaluate of Fairness for eMBB

We investigated eMBB allocation fairness under the proposed learning algorithm by calculating Jain's fairness index for each of the 800 test run, the results are shown in Fig. 11. Jain's fairness index for the j_{th} run, denoted as F_j, is a value ranging from 0 to 1 and is obtained as follows:

$$F_j = \left(\sum\nolimits_{u=1}^{U_{eMBB}} x_u\right)^2 / U_{eMBB} \cdot \sum\nolimits_{u=1}^{U_{eMBB}} x_u^2, \tag{67}$$

where U_{eMBB} represents the total number of eMBB users, and x_u represents the throughput for eMBB user $u \in \mathcal{U}$.

A higher mean fairness index closer to 1 indicates that, on average, the resource allocation is fair. A lower mean fairness index suggests that, on average, resource allocation tends to be less fair. Based on Fig. 11, the allocation of resources are relatively fair across most runs under our proposed CRS-DRL with the URLLC queue system.

H. The optimality performance of the proposed CRS-DRL

We also verified the optimality performance of the proposed CRS-DRL algorithms at different risk-level α. In particular, in Fig. 12, we have compared the performance of the proposed algorithm with the exhaustive search, for which each variable (i.e., the allocation factor and puncturing weights for each eMBB user) changed between 0 to 1 in step size of 0.005. We consider a scenario with two eMBB users sharing two resource blocks at each time-slot, with all other settings matching those outlined in Table II in Section VI.A. The results confirm that the average episodic reward under the proposed CRS-DRL algorithm closely approaches the global optimal solution achieved by the exhaustive search.

VII. Conclusions

To sum up, this paper studied the allocation of shared resources between eMBB and URLLC services. We proposed two deep reinforcement learning algorithms, i.e., the constrainted risk-neutral and risk-sensitive DRL algorithms, to tackle the scheduling challenge within a dynamic environment. The agent's objective is to maximize the normalized total throughput of eMBB services and the probability of not dropping URLLC packets within each mini-slot. We considered a constraint to ensure that the probability of unsuccessful transmission of URLLC packets remains below a predefined threshold, maintaining the statistical reliability of URLLC. Numerical results indicated that CRS-DRL, in comparison to CRN-DRL, reduces the tail end of the reward distribution, thereby mitigating significant losses and enhancing the system's ability to meet the stringent demands of both URLLC and eMBB services.

Appendix A
Proof of the Probability of Not Dropping URLLC Packets

The probability of not dropping URLLC packets can be expressed as

$$\begin{aligned} r_{\text{urllc}}^{\pi,\boldsymbol{s}} &= \mathbb{P}(D_m^{\pi,\boldsymbol{s}} = 0) \\ &= \sum_{q\in\mathbb{N}} P_Q^{\pi,\boldsymbol{s}}(q)\mathbb{P}(D_m^{\pi,\boldsymbol{s}} = 0|\mathcal{Q}_{m-1}^{\pi,\boldsymbol{s}} = q), \end{aligned} \tag{68}$$

where $\mathbb{P}(D_m^{\pi,\boldsymbol{s}} = 0|\mathcal{Q}_{m-1}^{\pi,\boldsymbol{s}} = q)$ denotes the conditional probability that the number of dropped packets is zero given the number of backlogged packets is $\mathcal{Q}_{m-1}^{\pi,\boldsymbol{s}} = q \in \mathbb{N}$, i.e.,

$$\mathbb{P}(D_m^{\pi,\boldsymbol{s}} = 0|\mathcal{Q}_{m-1}^{\pi,\boldsymbol{s}} = q) = \frac{\mathbb{P}(D_m^{\pi,\boldsymbol{s}} = 0 \cap \mathcal{Q}_{m-1}^{\pi,\boldsymbol{s}} = q)}{P_Q^{\pi,\boldsymbol{s}}(q)}. \tag{69}$$

If there is no dropped URLLC packet, the number of provided mini-slots for transmitting URLLC packets (i.e., $Y_m^{\pi,\boldsymbol{s}}$) should be no less than the URLLC queue backlog packets (i.e., $\mathcal{Q}_{m-1}^{\pi,\boldsymbol{s}}$). Thus,

$$\begin{aligned} \mathbb{P}(D_m^{\pi,\boldsymbol{s}} = 0 \cap \mathcal{Q}_{m-1}^{\pi,\boldsymbol{s}} = q) &= \mathbb{P}(C_m^{\pi,\boldsymbol{s}} \geq \mathcal{Q}_{m-1}^{\pi,\boldsymbol{s}}|\mathcal{Q}_{m-1}^{\pi,\boldsymbol{s}} = q) \\ &= \sum_{x=q}^{B} \mathbb{P}(Y_m^{\pi,\boldsymbol{s}} = x). \end{aligned} \tag{70}$$

Based on (68)-(70), the no-drop probability is updated as (8).

Authorized licensed use limited to: University of Warwick. Downloaded on June 18,2024 at 10:22:54 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2024.3373722

APPENDIX B
PROOF OF THE EXPECTED URLLC SERVED PACKETS

Let $P_{urllc}^{\pi,\boldsymbol{s},m}(x)$ denote $\mathbb{P}\big(U_{\text{urllc},m}^{\pi,\boldsymbol{s}} = x\big)$ where x represents the number of served URLLC packets, $x \in \{0,\dots,B\}$. Accordingly, the expected number of served URLLC packets in mini-slot m system state $\boldsymbol{s} \in \mathcal{S}$ under policy π can be expressed as

$$\mathbb{E}\big[U_{\text{urllc},m}^{\pi,\boldsymbol{s}}\big] = \sum\nolimits_{x=0}^{B} x \cdot P_{urllc}^{\pi,\boldsymbol{s},m}(x). \tag{71}$$

Therefore, the first step to find the expected value of served URLLC packets is deriving the PMF of served URLLC packets, i.e., $P_{urllc}^{\pi,\boldsymbol{s},m}(x)$. Considering (4), the event of $\big(U_{\text{urllc},m}^{\pi,\boldsymbol{s}} = x\big)$ occurs when either $(Y_m^{\pi,\boldsymbol{s}} = x)$ or $(\mathcal{Q}_{m-1}^{\pi,\boldsymbol{s}} + A_m = x)$. Consequently,

$$\begin{aligned} P_{urllc}^{\pi,\boldsymbol{s}}(x) = {} & \mathbb{P}(Y_m^{\pi,\boldsymbol{s}} = x)\mathbb{P}(\mathcal{Q}_{m-1}^{\pi,\boldsymbol{s}} + A_m \geq x) \\ & + \mathbb{P}(Y_m^{\pi,\boldsymbol{s}} > x)\mathbb{P}(\mathcal{Q}_{m-1}^{\pi,\boldsymbol{s}} + A_m = x), \forall x \in \mathcal{B}. \end{aligned} \tag{72}$$

The PMF of $(Y_m^{\pi,\boldsymbol{s}} = x)$ can be calculated by (3). The probability of $\mathbb{P}(\mathcal{Q}_{m-1}^{\pi,\boldsymbol{s}} + A_m = x)$ can be written as

$$\mathbb{P}(A_m + \mathcal{Q}_{m-1}^{\pi,\boldsymbol{s}} = x) = \sum_{a_1=0}^{x} P_a(a_1) \cdot P_Q^{\pi,\boldsymbol{s}}(x - a_1), \tag{73}$$

where $P_a(a_1) = \mathbb{P}(A_m = a_1)$ is the PMF that the number of the newly arrived URLLC packets is $A_m = a_1$, and $P_Q^{\pi,\boldsymbol{s}}(x - a_1)$ represents the steady state probability of the URLLC queuing backlog. In particular,

$$P_Q^{\pi,\boldsymbol{s}}(x - a_1) = \mathbb{P}(\mathcal{Q}_{m-1}^{\pi,\boldsymbol{s}} = x - a_1). \tag{74}$$

With the transition probability equation in (10), we can compute the steady-state probability of the queue based on (9). Having the steady-state probabilities of the queue, the PMF of the number of served packets can be derived based on (72), (3), and (73). Consequently, by substituting (72) into (71), $\mathbb{E}\big[U_{\text{urllc},m}^{\pi,\boldsymbol{s}}\big]$ can be found as (14).

REFERENCES

[1] P. Popovski et al, "Wireless access in ultra-reliable low-latency communication (URLLC)," *IEEE Transactions on Communications*, vol. 67, no. 8, pp. 5783–5801, 2019.

[2] K. Ovsthus, L. M. Kristensen *et al.*, "An industrial perspective on wireless sensor networks—a survey of requirements, protocols, and challenges," *IEEE communications surveys & tutorials*, vol. 16, no. 3, pp. 1391–1412, 2014.

[3] H. Ji, S. Park, J. Yeo, Y. Kim, J. Lee, and B. Shim, "Ultra-reliable and low-latency communications in 5G downlink: Physical layer aspects," *IEEE Wireless Communications*, vol. 25, no. 3, pp. 124–130, 2018.

[4] 3GPP, "Study on scenarios and requirements for next generation access technologies, version 15.0.0," *Sophia Antipolis, France*, p. Rep. TR 38.913, 2018.

[5] 3GPP, "Study on physical layer enhancements for nr ultra-reliable and low latency case (URLLC), version 16.0.0," *Sophia Antipolis, France*, p. Rep. TR 38.824, 2019.

[6] K. I. Pedersen, G. Berardinelli, F. Frederiksen, P. Mogensen, and A. Szufarska, "A flexible 5G frame structure design for frequency-division duplex cases," *IEEE Communications Magazine*, vol. 54, no. 3, pp. 53–59, 2016.

[7] S. Zhao, Y. Wang, T. Wang, and Z. Wang, "A reservation-based hybrid multiple access scheme for URLLC coexisting with eMBB," in *2021 IEEE/CIC International Conference on Communications in China (ICCC)*, 2021, pp. 916–921.

[8] K. I. Pedersen et al, "Punctured scheduling for critical low latency data on a shared channel with mobile broadband," in *2017 IEEE 86th Vehicular Technology Conference (VTC-Fall)*. IEEE, 2017, pp. 1–6.

[9] A. Anand, G. De Veciana, and S. Shakkottai, "Joint scheduling of URLLC and eMBB traffic in 5G wireless networks," *IEEE/ACM Transactions on Networking*, vol. 28, no. 2, pp. 477–490, 2020.

[10] M. Alsenwi, N. H. Tran, M. Bennis, A. K. Bairagi, and C. S. Hong, "eMBB-URLLC resource slicing: A risk-sensitive approach," *IEEE Communications Letters*, vol. 23, no. 4, pp. 740–743, 2019.

[11] A. K. Bairagi, M. S. Munir, M. Alsenwi, N. H. Tran, S. S. Alshamrani, M. Masud, Z. Han, and C. S. Hong, "Coexistence Mechanism Between eMBB and uRLLC in 5G Wireless Networks," *IEEE Transactions on Communications*, vol. 69, no. 3, pp. 1736–1749, 2021.

[12] Q. Chen, J. Wu, J. Wang, and H. Jiang, "Coexistence of URLLC and eMBB Services in MIMO-NOMA Systems," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 1, pp. 839–851, 2023.

[13] H. Yin, L. Zhang, and S. Roy, "Multiplexing URLLC traffic within eMBB services in 5G NR: Fair scheduling," *IEEE Transactions on Communications*, vol. 69, no. 2, pp. 1080–1093, 2021.

[14] H. Jang, J. Kim, W. Yoo, and J.-M. Chung, "URLLC mode optimal resource allocation to support HARQ in 5G wireless networks," *IEEE Access*, vol. 8, pp. 126 797–126 804, 2020.

[15] W. Zhang, M. Derakhshani, and S. Lambotharan, "Stochastic optimization of URLLC-eMBB joint scheduling with queuing mechanism," *IEEE Wireless Communications Letters*, vol. 10, no. 4, pp. 844–848, 2020.

[16] E. N. Tominaga et al, "Non-orthogonal multiple access and network slicing: Scalable coexistence of eMBB and URLLC," in *2021 IEEE 93rd Vehicular Technology Conference (VTC2021-Spring)*. IEEE, 2021, pp. 1–6.

[17] F. Saggese, M. Moretti, and P. Popovski, "Power Minimization of Downlink Spectrum Slicing for eMBB and URLLC Users," *IEEE Transactions on Wireless Communications*, vol. 21, no. 12, pp. 11 051–11 065, 2022.

[18] Y. Li et al, "Optimization of URLLC and eMBB multiplexing via deep reinforcement learning," in *2019 IEEE/CIC International Conference on Communications Workshops in China (ICCC Workshops)*. IEEE, 2019, pp. 245–250.

[19] J. Li and X. Zhang, "Deep reinforcement learning-based joint scheduling of eMBB and URLLC in 5G networks," *IEEE Wireless Communications Letters*, vol. 9, no. 9, pp. 1543–1546, 2020.

[20] Y. Huang, S. Li, C. Li, Y. T. Hou, and W. Lou, "A deep-reinforcement-learning-based approach to dynamic eMBB/URLLC multiplexing in 5G NR," *IEEE Internet of Things Journal*, vol. 7, no. 7, pp. 6439–6456, 2020.

[21] F. Saggese, L. Pasqualini, M. Moretti, and A. Abrardo, "eep Reinforcement Learning for URLLC data management on top of scheduled eMBB traffic," in *2021 IEEE Global Communications Conference (GLOBECOM)*, 2021, pp. 1–6.

[22] M. Alsenwi et al, "Intelligent resource slicing for eMBB and URLLC coexistence in 5G and beyond: A deep reinforcement learning based approach," *IEEE Transactions on Wireless Communications*, vol. 20, no. 7, pp. 4585–4600, 2021.

[23] P. Miskolczi, "Differences between mean-variance and mean-cvar portfolio optimization models," 2016.

[24] Q. Liu, N. Choi, and T. Han, "Constraint-Aware Deep Reinforcement Learning for End-to-End Resource Orchestration in Mobile Networks," in *2021 IEEE 29th International Conference on Network Protocols (ICNP)*, 2021, pp. 1–11.

[25] 3GPP, "New services and applications with 5G ultra-reliable low latency communication," *5G Americas WHITEPAPER*, November 2018.

[26] R. Serfozo, *Basics of applied stochastic processes*. Springer Science & Business Media, 2009.

[27] W. Zhang, M. Derakhshani, G. Zheng, C. S. Chen, and S. Lambotharan, "Bayesian optimization of queuing-based multichannel urllc scheduling," *IEEE Transactions on Wireless Communications*, vol. 22, no. 3, pp. 1763–1778, 2023.

[28] C. Gaskett, D. Wettergreen, and A. Zelinsky, "Q-learning in continuous state and action spaces," in *Australasian joint conference on artificial intelligence*. Springer, 1999, pp. 417–428.

[29] J. Ikemoto and T. Ushio, "Deep Reinforcement Learning Under Signal Temporal Logic Constraints Using Lagrangian Relaxation," *IEEE Access*, vol. 10, pp. 114 814–114 828, 2022.

[30] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic policy gradient algorithms," in *International conference on machine learning*. PMLR, 2014, pp. 387–395.

[31] T. P. Lillicrap et al, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.

[32] A. Tamar et al, "Optimizing the CVaR via sampling," in *Twenty-Ninth AAAI Conference on Artificial Intelligence*, 2015.

Authorized licensed use limited to: University of Warwick. Downloaded on June 18,2024 at 10:22:54 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2024.3373722

[33] Q. Yang et al, "Wcsac: Worst-case soft actor critic for safety-constrained reinforcement learning," in *Proceedings of the Thirty-Fifth AAAI Conference on Artificial Intelligence. AAAI Press, online*, 2021.
[34] Y. C. Tang, J. Zhang, and R. Salakhutdinov, "Worst cases policy gradients," *arXiv preprint arXiv:1911.03618*, 2019.
[35] M. G. Bellemare, W. Dabney, and R. Munos, "A distributional perspective on reinforcement learning," in *International Conference on Machine Learning*. PMLR, 2017, pp. 449–458.
[36] I. Olkin and F. Pukelsheim, "The distance between two random vectors with given dispersion matrices," *Linear Algebra and its Applications*, vol. 48, pp. 257–263, 1982.



Wenheng Zhang (Member, IEEE) received the BEng in Electrical and Electronic Engineering from the University of Ningbo China, Ningbo, China, in 2017. She furthered her education at Imperial College London, U.K., earning an M.Sc. in Communication and Signal Processing in 2018. She was awarded a Ph.D. in Signal Processing and Communications from the Wolfson School of Mechanical, Electrical and Manufacturing Engineering, Loughborough University, U.K. in 2023. Before completing her doctoral studies, she contributed as a Post-Doctoral Research Assistant within the same school from 2021 to 2022. Presently, she serves as a University Teacher specializing in digital communications at the Wolfson School of Mechanical, Electrical and Manufacturing Engineering, Loughborough University, U.K. Her research interests include the 5G and beyond mobile communications, including topics such as the scheduling problem, machine learning for wireless communications, resource allocation in wireless networks, signal processing, and optimization problems.



Mahsa Derakhshani Mahsa Derakhshani (Senior Member, IEEE) received the Ph.D. degree in electrical engineering degree from McGill University, Montréal, Canada, in 2013. She was an Honorary NSERC Postdoctoral Fellow with the Department of Electrical and Electronic Engineering, Imperial College London, from 2015 to 2016, a Post-Doctoral Research Fellow with the Department of Electrical and Computer Engineering, University of Toronto, Toronto, Canada, and a Research Assistant with the Department of Electrical and Computer Engineering, McGill University, from 2013 to 2015. She is currently a Senior Lecturer in digital communications with the Wolfson School of Mechanical, Electrical and Manufacturing Engineering, Loughborough University, U.K. Her research interests include machine learning and optimization for wireless communications, ultra-reliable low latency communications, edge computing, and Ambient IoT. She received several awards and fellowships, including Royal Academy of Engineering/The Leverhulme Trust Research Fellowship (2020–2021), the Natural Sciences and Engineering Research Council of Canada (NSERC) Post-Doctoral Fellow- ships (2015–2017), the Fonds de Recherche du Quebec–Nature et Technologies (FRQNT) Post-Doctoral Fellowship (2013–2015), the John Bonsall Porter Prize (2009–2010), and the McGill Engineering Doctoral Award (2008–2011). She serves as an Editor for IEEE WIRELESS COMMUNICATIONS LETTERS, IEEE Internet of Things Magazine, and IET Signal Processing journal.



Gan Zheng (Fellow, IEEE) received the BEng and the MEng from Tianjin University, Tianjin, China, in 2002 and 2004, respectively, in Electronic and Information Engineering, and the PhD degree in Electrical and Electronic Engineering from The University of Hong Kong in 2008. He is currently Professor in Connected Systems in the School of Engineering, University of Warwick, UK. His research interests include machine learning for wireless communications, reconfigurable intelligent surface, UAV communications and edge computing. He is the first recipient for the 2013 IEEE Signal Processing Letters Best Paper Award, and he also received 2015 GLOBECOM Best Paper Award, and 2018 IEEE Technical Committee on Green Communications & Computing Best Paper Award. He was listed as a Highly Cited Researcher by Thomson Reuters/Clarivate Analytics in 2019. He currently serves as an Associate Editor for IEEE Wireless Communications Letters and IEEE Transactions on Communications.



Sangarapillai Lambotharan (Senior Member, IEEE) is Professor of Signal Processing and Communications and the Director of the Institute for Digital Technologies at Loughborough University London. He received his Ph.D. degree in signal processing from Imperial College London, U.K., in 1997. He was a Visiting Scientist with the Engineering and Theory Centre, Cornell University, USA, in 1996. Until 1999, he was a Post-Doctoral Research Associate with Imperial College London. From 1999 to 2002, he was with the Motorola Applied Research Group, U.K., where he investigated various projects, including physical link layer modelling and performance characterization of 2.5G and 3G networks. He was with King's College London and Cardiff University as a Lecturer and a Senior Lecturer, respectively, from 2002 to 2007. His current research interests include 5G networks, MIMO, blockchain, machine learning, and network security. He has authored more than 270 journal articles and conference papers in these areas. He is a Fellow of IET and Senior Member of IEEE. He serves as a Senior Area Editor for the IEEE Transactions on Signal Processing and as an Associate Editor for the IEEE Transactions on Communications.

Authorized licensed use limited to: University of Warwick. Downloaded on June 18,2024 at 10:22:54 UTC from IEEE Xplore. Restrictions apply.
© 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.